

1. Serialization vs Deserialization

- **Data Serialization:** กระบวนการแปลง **Data Objects** จากโครงสร้างที่ซับซ้อนของเครื่องต้นทาง ให้ไปอยู่ในรูปแบบ **ข้อมูลมาตรฐานกลาง (ภาษากลาง)** ทั้งแบบ **Text** หรือ **Binary** เพื่อความเหมาะสมในการจัดเก็บและรับส่งข้อมูลข้ามระบบที่แตกต่างกัน.
- **Data Deserialization:** กระบวนการแปลงย้อนกลับ นำ **ภาษากลาง** ที่ได้รับจากต้นทางกลับมาเป็น **Object** รูปแบบเดิม บนเครื่องปลายทางเพื่อพร้อมใช้งาน.
- **รูปแบบการนำไปใช้งานจริง (Applications):**
 1. **Storing data into Databases:** แปลง Object (เช่น Java Object) ให้เป็น Byte Streams แล้วจัดเก็บลงฐานข้อมูล (เช่น MySQL) ผ่าน JDBC.
 2. **Transfer data to the network:** ส่ง Object ระหว่าง Client และ Server ใน Web/Mobile Applications.
 3. **Sharing data in a Distributed Object Model:** แบ่งปันข้อมูลระหว่างแพลตฟอร์มที่ต่างกันผ่าน Web Services ที่มี **SOAP** หรือ **REST** เป็นตัวกลาง.
 4. **Remote Method Invocation (RMI):** ส่ง Serialized Objects เป็น Parameter ไปทำงานในฟังก์ชันบนเครื่อง Remote Machine.
- **สาเหตุที่ต้องทำ:** คอมพิวเตอร์ต่างระบบกันมักมีความต่างด้าน ฮาร์ดแวร์, OS, โครงสร้างจัดเก็บข้อมูล และภาษาโปรแกรม จึงจำเป็นต้องมีรูปแบบข้อมูลมาตรฐานกลางที่ทุกระบบเข้าใจร่วมกัน

2. ข้อจำกัดของ HTML และหน้าเว็บแบบ Dynamic

- **ข้อจำกัดของ HTML:** HTML ถูกออกแบบมาเพื่อ **จัดรูปแบบและการนำเสนอเนื้อหา (Presentation)** เท่านั้น ไม่ได้ออกแบบมาเพื่อจัดเก็บข้อมูลให้ระบบอื่นนำไปประมวลผลต่อได้.
- **Static Web Page:** หน้าเว็บประเภทที่ผู้ใช้อ่านข้อมูลได้อย่างเดียว ไม่สามารถนำข้อมูลไปประมวลผลต่อฝั่ง Client ได้.
- **Dynamic Web Page:** หน้าเว็บที่ Client ได้รับข้อมูลจาก Server แล้วสามารถ **นำข้อมูลไปประมวลผลต่อหรือแสดงผลในรูปแบบใดก็ได้ตามต้องการอย่างอิสระ** หรือให้ผู้สร้าง Content บนหน้าเว็บเองได้.
- **ความจำเป็นของ Serialization Formats:** เพื่อสร้างหน้าเว็บแบบ Dynamic จึงต้องใช้ภาษากลางในการรับส่งข้อมูล เช่น XML, JSON, YAML แทน HTML

3. การเปรียบเทียบเชิงลึก: Text-based vs Binary-based

- **Text-based (XML, JSON, YAML):** มนุษย์สามารถเปิดอ่านและเข้าใจได้ง่าย (Human Readable).
- **Binary-based (BSON, MessagePack, ProtoBuf, Apache Thrift):** มนุษย์อ่านไม่เข้าใจโดยตรง.
- **จุดเปรียบเทียบ:** Binary format จะมี **ขนาดไฟล์เล็กกว่ามาก** และ **รับส่งข้อมูลได้เร็วกว่า** Text-based. แต่หากต้องการแลกเปลี่ยนข้อมูลผ่าน Web Services ที่มีอยู่เดิม แนะนำให้ใช้ **Text-based** เพราะเขียนและแปลผลบนหลากหลายแพลตฟอร์มได้ง่ายกว่า.

4. Milestones (ค.ศ. สำคัญที่ขอบออก True/False หรือเลือกตอบ)

- **ค.ศ. 2001:** กำหนดภาษา **JSON** โดย Douglas Crockford (และประกาศเป็นมาตรฐานสากล ค.ศ. 2017).
- **ค.ศ. 2004:** กำหนดภาษา **YAML** เวอร์ชันแรก (ซึ่งเป็น Extension ของ JSON).
- **ค.ศ. 2008:** Google เปิดตัว **Protocol Buffers (ProtoBuf)** อย่างเป็นทางการ.

- **ค.ศ. 2009:** เปิดตัว **BSON (Binary JSON)** เพื่อใช้เป็น Format จัดเก็บข้อมูลภายในฐานข้อมูล MongoDB.

5. ตารางเปรียบเทียบคุณสมบัติคู่เอก: XML vs JSON (ออกสอบเยอะมาก!)

คุณสมบัติ (Feature)	XML	JSON
ประเภทภาษา	Markup Language	Data Format (กระชับ สอดคล้องกับ JS Object)
โครงสร้างจัดเก็บ	Tree Structure	Key-Value Pairs
การใช้ Schema	มี Schema และ DTD ตรวจสอบความถูกต้องที่แข็งแกร่ง	ไม่มี Schema สนับสนุน ในการตรวจสอบโครงสร้าง
ความเร็ว (Speed)	ช้ากว่า (ขนาดไฟล์ใหญ่ ประมวลผลซับซ้อน)	เร็วกว่า (ขนาดไฟล์เล็ก ประมวลผลและผ่านแบนด์วิดท์ได้เร็ว)
Namespace & Comment	สนับสนุน	ไม่สนับสนุน
การรองรับ Array	ไม่สนับสนุนโดยตรง (ต้องใช้ Tag หุ้มข้อมูลย่อยทุกตัว)	สนับสนุนโครงสร้าง Array โดยตรง
ชนิดข้อมูล (Data Types)	หลากหลายมาก (รวมถึงกำหนดเองได้ / User-defined)	จำกัดเฉพาะ String, Number, Array, Boolean, Object, Null
ตัวแปลภาษา	ต้องพึ่งพา XML Parser โดยเฉพาะ	สามารถ Parse ด้วย JavaScript Function มาตรฐาน ได้ทันที

เมื่อไรควรเลือกใช้คืออะไร?

- **เลือก XML:** เหมาะกับงานที่ต้องการอธิบายข้อมูลละเอียดเพื่อการสืบค้น (มีเครื่องมือ XPath, DOM, XQuery, XSL ช่วย), งานที่เน้นตรวจสอบโครงสร้างแข็งแรงด้วย DTD/Schema หรือระบบความปลอดภัยสูงผ่านโปรโตคอล SOAP.
- **เลือก JSON:** เหมาะกับการรับส่งข้อมูลที่ต้องการความรวดเร็วผ่าน Rest APIs, การ Export ข้อมูลจากแอปไปลงฐานข้อมูล และเหมาะใช้ร่วมกับ JavaScript-based frameworks เช่น Node.js

ส่วนที่ 2: การแปลงข้อมูลจากตารางเป็น XML และ JSON

1. กฎเหล็ก "Well-Formed XML" (สำหรับชี้จุดผิดในโค้ด)

- **กฎการประกาศ (XML Declaration):** `<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>`
 - จุดสำคัญ: หากกำหนด `standalone="yes"` หมายถึงทำงานเดี่ยว ห้ามเรียกใช้ DTD อื่นร่วมด้วย ไม่เช่นนั้น Parser จะฟ้อง Error ทันที (ค่า Default คือ "no").
- **กฎการตั้งชื่อ Element (Element Names):**
 - ต้องขึ้นต้นด้วย: ตัวอักษร (Letter), Underscore (_), หรือ Colon (:) เท่านั้น.
 - ห้ามขึ้นต้นด้วย: ตัวเลข หรือ สัญลักษณ์พิเศษอื่น ๆ.
 - ห้ามมี: ช่องว่าง (Blank) หรือสัญลักษณ์พิเศษปนอยู่ในชื่อ Element.
 - ตัวอย่างที่ผิด: `<1Name>` (ขึ้นต้นด้วยตัวเลข), `<first name>` (มีช่องว่าง), `<first*name>` (มีเครื่องหมายพิเศษ).
- **กฎแท็กเปิด-ปิด (Tags matching):** ต้องเขียนเหมือนกันเป๊ะ และห้ามปิดสลับข้ามตำแหน่ง (Overlap).
 - ตัวอย่างที่ผิด (Overlap): `<Book><Name>XML Technology</Book></Name>` ❌.
 - ตัวอย่างที่ถูกต้อง: `<Book><Name>XML Technology</Name></Book>`.
- **กฎ Root Element:** เอกสาร XML ทั้งหมด จะต้อง มี Root Element เพียง 1 อันถ้วน คลุม Tag ย่อยทั้งหมดไว้.
 - ตัวอย่างที่ผิด: มี `<Book>...</Book>` วางลอย ๆ สองอันโดยไม่มีตัวคุมภายนอก ❌.

- ตัวอย่างที่ถูกต้อง: <Books> <Book>...</Book> <Book>...</Book> </Books>.

- **กฎ Empty Element:** เขียนได้สองรูปแบบคือ <student/> หรือ <student></student>.
- **กฎ Attribute:** จะต้องถูกกำหนดใน Tag เปิด อยู่หลังเครื่องหมาย = และค่าของ Attribute **ต้องหุ้มด้วยเครื่องหมายคำพูดคู่ " "** เสมอ.
 - ตัวอย่าง: <STUDENT id="ST949318" />.

2. การแปลง Element ➔ Attribute (ข้อสอบชอบให้ออกแบบสลับไปมา)

- โครงสร้างแบบ Element ซ้อน:

```
<employee>
  <id>111111</id>
  <name>Somchai</name>
</employee>
```

- แปลงเป็นรูปแบบ Attribute:

```
<employee id="111111">Somchai</employee>
```

ส่วนที่ 3: เทคนิคการเขียนโค้ด JavaScript ดึงข้อมูล JSON และจัดการ DOM

1. การเข้าถึงข้อมูลใน JSON Object & Array

- **JSON Object:** ข้อมูลต้องเป็นคู่ "key": "value" คีย์และค่าที่เป็น String ต้องอยู่ในเครื่องหมายคำพูดคู่ "" เสมอ และห่อด้วย {}.
- **JavaScript Object:** แตกต่างจาก JSON ตรงที่ **Key ไม่จำเป็นต้องใส่เครื่องหมายคำพูดหุ้ม** เช่น {name: "Willy"}.
- **สูตรการดึงข้อมูล:**
 - เข้าถึง property ของ Object: objectName.propertyName หรือ objectName["propertyName"].
 - เข้าถึงสมาชิกของ Array: ใช้ดัชนีระบุช่อง arrayName[index] (เริ่มนับตัวแรกจาก 0).
- ตัวอย่างการไถ่โครงสร้างซับซ้อน (Nested JSON Access):

```
const txt = ' { "id": 1, "name": "Leanne Graham", "address": { "street": "Kulas Light", "city": "Gorouh" } } ';
```

```
const obj = JSON.parse(txt); //
```

```
// ดึงค่าถนน: obj.address.street
```

```
// ดึงค่าเมือง: obj.address.city
```

2. สรุปคำสั่ง DOM Properties & Methods สำหรับข้อสอบเดิมคำ

- **JSON.parse(jsontxt):** ใช้แปลง JSON String (ข้อความดิบจาก Server) ให้กลายเป็น JavaScript Object พร้อมใช้งาน.
- **xmlDoc.documentElement:** ชี้ไปยัง Root Element หลักของเอกสาร.
- **childNodes[i]:** ชี้ไปยังโหนดลูกลำดับที่ i (นับเริ่มต้นจาก 0).
- **firstChild:** เป็น Property ทางลัด ใช้ชี้ไปยังโหนดลูกตัวแรกสุด (มีค่าเท่ากับ childNodes).
- **nodeValue:** ดึงหรือกำหนดค่าข้อความที่อยู่ภายใน Text Node นั้น ๆ.
 - ตัวอย่างโจทย์ไล่ชั้นเพื่อดึงชื่อคน (จากรูปภาพในบทเรียน): xmlDoc.documentElement.childNodes.childNodes.no deValue (จะดึงค่า "Alan" ออกมา).
- **getElementsByTagName("tag_name"):** ค้นหา Elements ทั้งหมดตามชื่อแท็ก ผลลัพธ์ที่ได้จะเป็น Array ต้องระบุดัชนีช่อง เช่น `` หรือ .
 - ตัวอย่างดึงค่าแท็ก lastName ตัวที่ 2: xmlDoc.getElementsByTagName("lastName").firstChild .nodeValue (จะดึงค่า "Li" ออกมา).

- **getElementsByClassName("class_name"):** ดึงกลุ่ม Elements ตามคลาสที่ระบุ คืนค่ากลับมาเป็น Array.

- **การสร้าง Element และนำไปต่อท้าย (Append):**

1. สร้าง Element Node ใหม่: const newP = document.createElement("p");.
2. สร้าง Text Node ใหม่: const txtNode = document.createTextNode("Hello World");.
3. นำ Text Node ไปใส่ใน Element Node: newP.appendChild(txtNode);.
4. ผลลัพธ์ที่ได้คือโครงสร้าง: <p>Hello World</p>.

ส่วนที่ 4: การแปลงความสัมพันธ์ตารางเป็น JSON สำหรับเก็บใน MongoDB

ใน MongoDB ข้อมูลแต่ละเรคคอร์ดเรียกว่า **Document** และเก็บข้อมูลในรูปแบบ JSON. การออกแบบโครงสร้างความสัมพันธ์มี 2 รูปแบบหลัก:

1. ความสัมพันธ์แบบ One-to-One (1:1)

- **Reference Model (โมเดลอ้างอิง):** เก็บแยกเป็น 2 Collections เช่น Student และ PermanentAddress โดยอ้างอิงด้วย ID.
 - ข้อเสีย: การ Query จะต้องเชื่อมโยงข้อมูลข้ามเอกสาร ทำให้ระบบซับซ้อนและใช้เวลาสืบค้นค่อนข้างนาน.
- **Embedded Document Model (โมเดลฝังตัว - แนะนำ):** ยุบรวมเอา PermanentAddress มาฝังตัวเป็น Object ซ่อนอยู่ข้างใน Document ของ Student โดยตรง.
 - โครงสร้าง JSON:

```
{
  "studentID": "S00001",
  "studentName": "Willy",
  "permanentAddress": {
    "number": "999-1",
    "city": "LA",
    "postcode": 28788
  }
}
```

2. ความสัมพันธ์แบบ One-to-Many (1:M)

- **Embedded Document Model (Array of Objects - มักออกสอบบ่อยที่สุด):** ยุบรวมข้อมูลตารางที่สัมพันธ์กันมาใส่ในอาร์เรย์ (Array) ของ Object ย่อยภายในเอกสารเดียว.
 - ข้อดี: เขียน Query ค้นหาข้อมูลได้ง่ายมาก ไม่ต้องทำคำสั่ง Join ข้อมูล และประมวลผลดึงข้อมูลได้รวดเร็วที่สุด.
 - โครงสร้าง JSON:

```
{
  "studentID": "S00001",
  "studentName": "Willy",
  "major": "CS",
  "address": [
    { "number": "999-1", "city": "LA", "postcode": 28788 },
    { "number": "555-8", "city": "NY", "postcode": 86777 }
  ]
}
```

3. ความสัมพันธ์แบบ Many-to-Many (N:M)

- **Reference Model:** ตัวอย่าง เช่น นักศึกษาหลายคนสามารถลงทะเบียนเรียนได้หลายวิชา (Students กับ Courses ผ่านตารางเชื่อม

- Enrollments). บน MongoDB จะออกแบบโดยใช้ฟิลด์อ้างอิงและใช้ฟังก์ชันคำนวณขั้นสูง.
- ข้อดี: สามารถสืบค้นข้อมูลได้สองทิศทางพร้อมกันได้อย่างสะดวก (สืบค้นว่านักศึกษาแต่ละคนลงเรียนวิชาอะไรบ้าง และวิชาแต่ละวิชามักมีนักศึกษาคนไหนมาเรียนบ้าง).
 - ข้อเสีย: ตอนทำการเรียกค้นหาข้อมูลแบบรวม จะต้องรันคำสั่ง \$lookup เพื่อทำการ JOIN เชื่อมต่อข้อมูลระหว่างคอลเลกชัน

OrderID	Customer	Item	Qty
O01	Willy	Pen	2
O01	Willy	Book	3

```
[
  {
    "OrderID": "O01",
    "Customer": "Willy",
    "Items": [
      {
        "ItemName": "Pen",
        "Qty": 2
      },
      {
        "ItemName": "Book",
        "Qty": 3
      }
    ]
  }
]
```

รูปแบบที่ 1: การแปลง JSON String และเข้าถึงข้อมูลแบบ Nested (อิงจากแบบฝึกหัดท้ายบท บทที่ 3)

อาจารย์มักจะให้ JSON String ที่มีโครงสร้างซับซ้อนมา แล้วให้เติมโค้ด JavaScript เพื่อดึงค่าไปใส่ในแท็ก HTML ให้ถูกต้อง

➔ โจทย์จำลอง:

```
<h1 id="bookname"></h1>
<h3 id="author"></h3>
```

```
<script>
// JSON String ข้อมูลหนังสือ
const bkk = ' { "id": 1, "name": "Json Applications", "author": { "fname": "John", "lname": "Markov" } }';

// 1. แปลง JSON String ให้เป็น JavaScript Object
const bk = _____(1)_____(bkk);

// 2. อ้างอิง HTML Element ด้วย DOM Method
const bkname =
document._____(2)_____("bookname");
const auname = document._____(3)_____("author");

// 3. ดึงข้อมูลใน Object ไปแสดงผลใน HTML (ระวังการได้ชั้นข้อมูลซ้อน)
bkname.textContent = "My favorite book is " +
_____(4)_____;
```

```
auname.textContent = "Author is " + _____(5)_____ + "
" + _____(6)_____;
```

➔ เฉลยคำตอบ:

- (1) JSON.parse (แปลง JSON string ให้เป็น Object)
- (2) getElementById (ดึงแท็ก id="bookname")
- (3) getElementById (ดึงแท็ก id="author")
- (4) bk.name (เข้าถึงคีย์ name ของออบเจกต์ bk)
- (5) bk.author.fname (เข้าถึง fname ที่ซ่อนอยู่ใต้ author)
- (6) bk.author.lname (เข้าถึง lname ที่ซ่อนอยู่ใต้ author)

รูปแบบที่ 2: การเข้าถึงข้อมูล XML DOM ด้วย Properties และ Text Node (อิงจากบทที่ 2-2)

จุดดักคะแนนที่ใหญ่ที่สุดในข้อสอบทฤษฎี! คือความเข้าใจเรื่อง **Text Node** (ข้อความที่อยู่ด้านในแท็ก เช่น <firstName>Alan</firstName> คำว่า "Alan" คือ Text Node ซึ่งเป็น child Node ของ firstName อีกทีหนึ่ง)

➔ โจทย์จำลอง:

กำหนดตัวแปร XML Object ชื่อ xmlDoc ที่เก็บโครงสร้างดังนี้:

```
<persons>
  <person>
    <firstName>Alan</firstName>
    <lastName>Wilkes</lastName>
  </person>
  <person>
    <firstName>Howard</firstName>
    <lastName>Li</lastName>
  </person>
</persons>
```

```
// กรณีที่ 1: เข้าถึงคำว่า "Alan" โดยใช้ DOM Properties (ไต่โหนดตามลำดับชั้น)
// persons คือ root element
const root = xmlDoc._____(1)_____;
```

```
// ดึงคำว่า "Alan" (เป็นลูกคนแรกของ person คนแรก)
const name1 =
root.childNodes._____(2)_____, _____(3)_____
_____.nodeValue;
```

```
// กรณีที่ 2: เข้าถึงคำว่า "Li" (lastName คนที่สอง) โดยใช้ DOM Method ร่วมกับ Property
const name2 =
xmlDoc._____(4)_____("lastName")[12]._____(5)_____
_____._____(6)_____;
```

```
// กรณีที่ 3: ใช้ Property ทางเลือกพิเศษ แทนการดึง firstChild.nodeValue
const name3 =
xmlDoc.getElementsByTagName("lastName")[12]._____(7)_____
_____;
```

➔ เฉลยคำตอบ:

- (1) documentElement (ดึง Root element ของเอกสาร)
- (2) childNodes (ลูกคนแรกของ person คือ firstName)
- (3) firstChild หรือ childNodes (ชี้ลงไปยัง Text Node ที่เก็บค่า "Alan")

- (4) `getElementsByTagName` (ค้นหาจากชื่อแท็ก)
- (5) `firstChild` (ดึง Text Node ของ `lastName` คนที่สอง)
- (6) `nodeValue` (ดึงค่าข้อความข้างใน Text Node ออกมา ซึ่งจะได้ "Li")
- (7) `textContent` (Property ทางเลือก ดึงข้อความได้ทันทีโดยไม่ต้องผ่าน Text Node)

รูปแบบที่ 3: การเปลี่ยน Attribute และ Style ของ HTML (อิงจากบทที่ 2-2)
 โจทย์มักจะสั่งให้ทำการเปลี่ยนรูปภาพ เปลี่ยนขนาดช่องอินพุต หรือเปลี่ยนสีพื้นหลังผ่าน DOM Properties/Methods

→ โจทย์จำลอง:

```

<input type="text" id="inputText" size="50" />
<p class="example">A paragraph 1</p>
<p class="example">A paragraph 2</p>

<script>
// 1. เปลี่ยนรูปภาพ img1 เป็น "images/new.jpg" (เดิมได้ 2 วิธี)
// วิธีที่ 1: ใช้ Property โดยตรง
document.getElementById("img1")._____ (1) _____ =
"images/new.jpg";
// วิธีที่ 2: ใช้ Method
document.getElementById("img1")._____ (2) _____ ("src",
"images/new.jpg");

// 2. เปลี่ยนขนาด (size) ของ inputText เป็น "30"
document.getElementById("inputText").setAttribute(_____ (3) _____, "30");

// 3. เปลี่ยนสีพื้นหลังของ "A paragraph 1" ให้เป็นสีแดง (ใช้ querySelectorAll)
const nodeList =
document._____ (4) _____ ("p.example");
nodeList._____ (5) _____ ._____ (6) _____
__ = "red";

// 4. เปลี่ยนสีตัวอักษรของ "A paragraph 2" ให้เป็นสีน้ำเงิน (ใช้
getElementsByClassName)
const classList = document._____ (7) _____ ("example");
classList[12].style._____ (8) _____ = "blue";
</script>
```

→ เฉลยคำตอบ:

- (1) `src`
- (2) `setAttribute`
- (3) `"size"`
- (4) `querySelectorAll`
- (5) `style`
- (6) `backgroundColor` (ใน JS สไตล์ที่มีขีดกลาง เช่น `background-color` จะเขียนแบบ CamelCase)
- (7) `getElementsByClassName`
- (8) `color`

รูปแบบที่ 4: การสร้างและเพิ่ม Element ใหม่ลงใน DOM (อิงจากบทที่ 2-2)

การสร้างแท็ก HTML ใหม่ขึ้นมาในหน่วยความจำ แล้วนำไปต่อท้ายหรือแทรกในจุดที่ต้องการ

→ โจทย์จำลอง:

```
<div id="div1">
  <p id="p1">First paragraph.</p>
</div>

<script>
// ต้องการสร้างแท็ก <p>This is new.</p> แล้วนำไปต่อท้ายใน div1
const newpara = document._____ (1) _____ ("p"); // 1.
สร้าง Element Node
const newText = document._____ (2) _____ ("This is
new."); // 2. สร้าง Text Node

newpara._____ (3) _____ (newText); // 3. นำ Text Node
ใส่ใน Element Node

const parent = document.getElementById("div1");
parent._____ (4) _____ (newpara); // 4. นำ Element
Node ไปต่อท้าย div1
</script>
```

→ เฉลยคำตอบ:

- (1) `createElement` (สร้าง Element Node)
- (2) `createTextNode` (สร้าง Text Node สำหรับเก็บเนื้อหาข้อความ)
- (3) `appendChild` (นำโหนดลูกไปเชื่อมต่อกับโหนดแม่)
- (4) `appendChild` (ต่อท้ายเข้ากับ parent element)

รูปแบบที่ 5: การทำงานกับ AJAX & REST API (อิงจากบทที่ 3-2)
 การเขียนโค้ดเรียกใช้งาน REST API หลังบ้านทั้ง 3 วิธีหลัก (ออกข้อสอบเขียนฟังก์ชันบ่อยมาก!)

→ โจทย์จำลอง:

เติมค่าในช่องว่างเพื่อทำการยิง Request ไปยัง Server และแปลงค่าที่ได้กลับมา
`const myapi = "http://localhost/employee/api.php?eid=1111";`

// วิธีที่ 1: AJAX แบบดั้งเดิม (XMLHttpRequest)

```
function loadAJAX() {
  const xhttp = new _____ (1) _____ ();
  xhttp._____ (2) _____ = function() {
    if (this.readyState == 4 && this.status == 200) {
      // ดึงผลลัพธ์ที่เป็น Text มาแปลงเป็น Object
      const obj =
        _____ (3) _____ ._____ (4) _____ (this.re
sponseText);
      console.log(obj.ename);
    }
  };
  xhttp.open("GET", myapi, true);
  xhttp.send();
}
```

// วิธีที่ 2: Promise + Fetch API

```
function loadFetch() {
    _____(5)_____ (myapi)
    .then(response => response. _____(6)_____()) //
    แปลงผลลัพธ์เป็น Object
    .then(myObj => {
        console.log(myObj.ename);
    })
    .catch(err => console.log(err));
}

// วิธีที่ 3: Async & Await
_____ (7) _____ function loadAsync() {
    try {
        const response = _____(8)_____ fetch(myapi);
        const myObj = await response. _____(9)_____();
        console.log(myObj.ename);
    } catch(error) {
        console.log(error.message);
    }
}
```

- ➔ เฉลยคำตอบ:
- (1) XMLHttpRequest
 - (2) onreadystatechange (Event ดักฟังสถานะ)
 - (3) JSON
 - (4) parse (รวมกันเป็น JSON.parse)
 - (5) fetch
 - (6) json (ใน fetch ต้องเรียกใช้ .json() เสมอเพื่อคืนค่ากลับมาเป็น Promise)
 - (7) async (ระบุหน้าฟังก์ชันเพื่อใช้งาน await ข้างใน)
 - (8) await (สั่งให้หยุดรอผลลัพธ์)
 - (9) json (แปลงข้อมูลในรูปแบบ JSON)

หมวดที่ 1: การแปลงข้อมูลจากโครงสร้างตารางเป็น XML และ JSON (รวมถึง Well-Formed XML)

โจทย์ที่ 1: การแปลงข้อมูลตารางระดับซับซ้อนเป็น XML และ JSON + Tree Structure

ตารางข้อมูลสั่งซื้อ (Orders): | OrderID (Attribute) | CustomerName | OrderDate | ItemName | Qty | Price | | :--- | :--- | :--- | :--- | :--- | :--- | | ORD-99 | Mr. Yue | 20/11/2006 | USB | 2 | €10 | | ORD-99 | Mr. Yue | 20/11/2006 | A4 Paper | 10 | €5 |

- คำสั่งที่ 1.1: จงแปลงตารางด้านบนให้เป็นเอกสาร XML ที่ถูกต้องตามกฎ Well-Formed โดยกำหนดให้ OrderID เป็น Attribute และให้ทำการยุบรวมข้อมูลรายการสินค้าที่สั่งซื้อ (Items) ภายใต้ผู้สั่งซื้อคนเดียวกัน
- คำสั่งที่ 1.2: จงเขียนแผนผัง XML Tree Structure ให้สอดคล้องกับเอกสาร XML ในข้อ 1.1
- คำสั่งที่ 1.3: จงแปลงตารางข้อมูลสั่งซื้อนี้เป็น JSON ในรูปแบบ Array of Objects

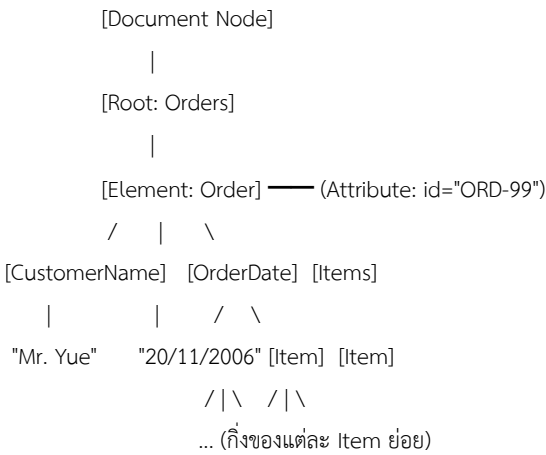
เฉลยคำตอบและโครงสร้าง:

เฉลย 1.1 (โค้ด XML):

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<Orders>
```

```
<Order id="ORD-99">
  <CustomerName>Mr. Yue</CustomerName>
  <OrderDate>20/11/2006</OrderDate>
  <Items>
    <Item>
      <ItemName>USB</ItemName>
      <Qty>2</Qty>
      <Price>€10</Price>
    </Item>
    <Item>
      <ItemName>A4 Paper</ItemName>
      <Qty>10</Qty>
      <Price>€5</Price>
    </Item>
  </Items>
</Order>
</Orders>
```

เฉลย 1.2 (XML Tree Structure):



เฉลย 1.3 (โค้ด JSON):

```
[
  {
    "OrderID": "ORD-99",
    "CustomerName": "Mr. Yue",
    "OrderDate": "20/11/2006",
    "Items": [
      {
        "ItemName": "USB",
        "Qty": 2,
        "Price": "€10"
      },
      {
        "ItemName": "A4 Paper",
        "Qty": 10,
        "Price": "€5"
      }
    ]
  }
]
```

โจทย์ที่ 2: ซ้ำจุดผิดและแก้ไขเอกสาร XML ให้เป็น "Well-Formed XML"

โจทย์: จงวงกลมชี้จุดผิดอย่างน้อย 5 จุดในเอกสาร XML ต่อไปนี้ และเขียนเอกสารที่แก้ไขแล้วให้เป็น Well-Formed Document:

```
<?xml version="1.0" ?>

<Person>

  <Name title="Mr">

    <First>Vernon<Last>Callaby</First></Last>

  <Title>Office Manager</Title>

  <Phone Ext>582</Phone Ext>

  <eMail>v.callaby@nanonull.com<eMail>

  <#Shares>1500</#Shares>

</person>
```

เฉลยคำตอบและการวิเคราะห์จุดผิด:

- จุดที่ 1: เครื่องหมายคำพูดของ attribute title="Mr" เป็นเครื่องหมายคำพูดแบบโค้งสไตร์จัดรูปแบบอักษร ซึ่งผิดหลักไวยากรณ์ XML ต้องใช้เครื่องหมายคำพูดตรง "Mr" เท่านั้น.
- จุดที่ 2 (Overlapping): แท็กปิด<First>Vernon<Last>Callaby</First></Last> มีการซ้อนทับกันข้ามลำดับชั้น ต้องแก้เป็น<First>Vernon</First><Last>Callaby</Last>.
- จุดที่ 3 (ชื่อ Element มีช่องว่าง): <Phone Ext> มีช่องว่างอยู่กลางชื่อ Element ซึ่งผิดกฎ ต้องแก้เป็น <PhoneExt> หรือ <Phone_Ext>.
- จุดที่ 4 (แท็กปิดไม่มีเครื่องหมาย /): แท็กปิดของ <eMail>...</eMail> เขียนสะกดเป็น <eMail> ซ้ำ (ลืมเครื่องหมาย /) ต้องแก้เป็น </eMail>.
- จุดที่ 5 (ชื่อ Element ขึ้นต้นด้วยสัญลักษณ์พิเศษ): <#Shares> ใช้เครื่องหมาย # นำหน้าชื่อ ซึ่งผิดกฎการตั้งชื่อ ต้องแก้เป็น <Shares>.
- จุดที่ 6 (Case Sensitive): แท็กเปิดเป็นอักษรพิมพ์ใหญ่ <Person> แต่แท็กปิดด้านล่างสุดกลับเป็นอักษรพิมพ์เล็ก </person> ต้องจับคู่ตัวพิมพ์ให้เหมือนกันเป๊ะ เช่น </Person>.

โค้ด XML ที่แก้ไขถูกต้องแล้ว:

```
<?xml version="1.0" encoding="UTF-8" ?>

<Person>

  <Name title="Mr">

    <First>Vernon</First>

    <Last>Callaby</Last>

  </Name>

  <Title>Office Manager</Title>

  <Phone_Ext>582</Phone_Ext>

  <eMail>v.callaby@nanonull.com</eMail>

  <Shares>1500</Shares>

</Person>
```

หมวดที่ 2: การเติมโค้ด JavaScript & DOM (เน้นการเข้าถึง JSON และจัดการ HTML DOM)

โจทย์ที่ 3: การไล่สายตาเข้าถึง Nested JSON Object & Array

กำหนดตัวแปร JSON String ชื่อ jsontxt:

```
const jsontxt = ' { "company": "Sun Star", "numberEmp": 10000, "ceo": { "name": "Mary Jane", "age": 45 }, "address": { "state": "NY", "geo": { "lat": "40.7128 N", "long": "74.0060 W" } }, "offices": ["Building A", "Building B"] } ';
```

จงเติมคำสั่งในช่องว่างเพื่อให้โค้ดสามารถดึงข้อมูลและแสดงผลทาง Console ได้ถูกต้อง:

```
const obj = _____(1)_____(jsontxt);
```

```
console.log("CEO Name: " + _____(2)_____); // ต้องการ
แสดง: Mary Jane

console.log("Latitude: " + _____(3)_____); // ต้องการ
แสดง: 40.7128 N

console.log("Main Office: " + _____(4)_____); // ต้องการ
แสดง: Building B

เฉลยคำตอบ:
```

- (1) JSON.parse (แปลง string เป็น object)
- (2) obj.ceo.name (เข้าถึงฟิลด์ name ภายใต้อ็อบเจกต์ ceo)
- (3) obj.address.geo.lat (ไล่ขึ้นจาก address → geo → lat)
- (4) obj.offices (เข้าถึงสมาชิกตัวที่ 2 ของ Array offices เริ่มนับจากดัชนี 0)

โจทย์ที่ 4: การสืบค้นข้อมูลและเข้าถึง Text Node ของ XML DOM (จุดดักคะแนน!) กำหนดโครงสร้างไฟล์ XML ในหน่วยความจำผ่านตัวแปร xmlDoc ดังนี้:

```
<customers>

  <customer id="C01">

    <name>John Wilkey</name>

    <address><city>Madison</city></address>

  </customer>

  <customer id="C02">

    <name>Billy Mac</name>

    <address><city>Banheim</city></address>

  </customer>

</customers>
```

จงเติมโค้ด JavaScript ลงในช่องว่างเพื่อดึงค่าผลลัพธ์ออกมาใน Popup Alert ให้ถูกต้อง:

```
// 1. ต้องการดึงชื่อผู้ใช้คนแรก "John Wilkey" ผ่านการใช้ Properties โหนดลูกตามลำดับชั้น

const root = xmlDoc._____(1)_____; // ดึง root element

const name1 =

root.childNodes._____(2)_____._____(3)_____.nodeValue;

alert(name1); // แสดงป๊อปอัป "John Wilkey"
```

```
// 2. ต้องการดึงชื่อเมืองของลูกค้าคนที่สอง "Banheim" ผ่านการใช้ getElementsByTagName

const cityTag2 = xmlDoc._____(4)_____("city")[10];

const cityName =

cityTag2._____(5)_____._____(6)_____

____;

alert(cityName); // แสดงป๊อปอัป "Banheim"
```

```
// 3. ต้องการ(Property ทางเลือก)รวบโดยไม่ต้องอ้างอิง Text Node เพื่อดึงชื่อเมืองคนที่สอง "Banheim"

const cityNameShortcut =

xmlDoc.getElementsByTagName("city")[10]._____(7)_____

____;

alert(cityNameShortcut);

เฉลยคำตอบ:
```

- (1) documentElement (ตั้งแท็ก root <customers>)
- (2) childNodes (ลูกตัวแรกของ <customer> คือ <name>)
- (3) firstChild (หรือ childNodes) (แทรกเพื่อเข้าถึง Text Node ที่อยู่ภายใต้แท็ก <name>)
- (4) getElementsByTagName (สืบค้นโหนดด้วยชื่อแท็ก)
- (5) firstChild (หรือ childNodes) (ชี้เข้าสู่ตัวอักษร "Banheim" ซึ่งเป็น Text Node)
- (6) nodeValue (ดึงข้อความดิบออกมาจาก Text Node)
- (7) textContent (ดึงข้อความภายใน Element Node ออกมาได้ทันทีโดยไม่ต้องผ่าน Text Node)

โจทย์ที่ 5: การสร้างและแก้ไขหน้าเว็บ Dynamic ด้วย DOM Properties & Methods

กำหนดโครงสร้าง HTML:

```
<div id="item-box">
  <p class="item-title">Product List</p>
  <ul class="list-group">
    <li class="list-item">Pen</li>
    <li class="list-item">Notebook</li>
  </ul>
</div>
```

จงเติมคำสั่งในช่องว่างเพื่อให้โค้ด JavaScript ทำงานสำเร็จตามเป้าหมาย:

// 1. เปลี่ยนข้อความจากคำว่า "Pen" เป็น "Pencil" ผ่านทาง Class Name

```
const items = document. (1) ("list-item");
items. (2) = "Pencil";
```

// 2. ปรับแต่งสไตล์ให้ "Notebook" (รายการลำดับที่ 2) ตัวอักษรเป็นสีแดงและขนาดใหญ่ขึ้น

```
items[10]. (3) . (4)
___ = "red";
items[10].style. (5) = "20px";
```

// 3. สร้างแท็ก <li class="list-item">Eraser ใหม่ขึ้นมา แล้วนำไปแทรก

ต่อท้าย ข้างบน

```
const newLi = document. (6) ("li"); // สร้าง
โหนด Element
const newText = document. (7) ("Eraser"); //
สร้างโหนดข้อความ
```

newLi. (8) (newText); // เชื่อมข้อความเข้ากับ
Element ใหม่

newLi. (9) ("class", "list-item"); // กำหนด

คุณลักษณะคลาสให้เรียบร้อย

```
const listContainer = document.querySelector(".list-group");
```

```
listContainer. (10) (newLi); // นำไปต่อท้ายลิสต์
ทั้งหมด
```

เฉลยคำตอบ:

- (1) getElementsByClassName
- (2) textContent (หรือ innerHTML)
- (3) style

- (4) color
- (5) fontSize (ระวังตัวพิมพ์ใหญ่ S)
- (6) createElement
- (7) createTextNode
- (8) appendChild
- (9) setAttribute
- (10) appendChild

โจทย์ที่ 6: การสืบค้น REST API หลังบ้านด้วย Promise vs Async-Await

โจทย์: จงเขียนหรือเติมเต็มโค้ด JavaScript เพื่อสืบค้นข้อมูลจาก URL:

http://localhost/api/employee.php?eid=101 ด้วยสองวิธีการหลัก:

เฉลยคำตอบและเติมเพลงตามมาตรฐาน (จดเข้าห้องสอบได้เลย!):

แบบที่ 1: เติมคำสั่งของ Promise (.then chain)

```
function getEmployeeData() {
  const apiURL = "http://localhost/api/employee.php?eid=101";

  _____ (1) _____ (apiURL)
    .then(response => {
      // แปลง JSON string เป็น JS Object
      return response. _____ (2) _____ ();
    })
    .then(myObj => {
      // แสดงข้อมูลพนักงานชื่อ ename ลงบนหน้าจอผ่าน DOM
      document.getElementById("emp-name").textContent =
myObj.ename;
    })
    .catch(err => {
      console.log("Error: " + err. _____ (3) _____ );
    });
}
```

- (เฉลย): (1) fetch | (2) json | (3) message

แบบที่ 2: เติมคำสั่งของ Async & Await

```
_____ (1) _____ function getEmployeeDataAsync() {
  const apiURL = "http://localhost/api/employee.php?eid=101";

  try {
    // รอรับคำตอบแรกจาก Server
    const response = _____ (2) _____ fetch(apiURL);

    // รอการแปลงค่า json string ออกมาเป็น JavaScript Object
    const myObj = await response. _____ (3) _____ ();

    document.getElementById("emp-name").textContent =
myObj.ename;
  } catch (error) {
    console.error("Error occurred:", error.message);
  }
}
```

- (เฉลย): (1) async | (2) await | (3) json

หมวดที่ 3: การแปลงโครงสร้างตารางเป็น JSON ที่สอดคล้องกับที่เก็บใน MongoDB

โจทย์ที่ 7: การจัดเก็บความสัมพันธ์ One-to-Many (1:M) (อิงจากแบบฝึกหัด Review 1)

ตารางต้นทาง: ตาราง Departments (ฝั่ง 1): | deptNo (PK) | deptName | tel | | :--- | :--- | :--- | | d01 | Computer Support | 08555554 |

ตาราง Employees (ฝั่ง M): | empID (PK) | empName | email | deptNo (FK) | | :--- | :--- | :--- | :--- | | e0001 | Smith Loy | smith@gmail.com | d01 | | e0002 | Mary Jane | mar@hotmail.com | d01 |

โจทย์ประยุกต์: จงแปลงโครงสร้างตารางความสัมพันธ์ One-to-Many แบบฐานข้อมูลเชิงสัมพันธ์ (RDB) นี้ ให้กลายเป็นเอกสาร Embedded Document JSON สำหรับฐานข้อมูล MongoDB

เฉลยโครงสร้าง JSON (ฝั่งพนักงานไว้ได้แผนก):

```
{
  "_id": "d01",
  "deptName": "Computer Support",
  "tel": "08555554",
  "employees": [
    {
      "empID": "e0001",
      "empName": "Smith Loy",
      "email": "smith@gmail.com"
    },
    {
      "empID": "e0002",
      "empName": "Mary Jane",
      "email": "mar@hotmail.com"
    }
  ]
}
```

- คำอธิบายเพิ่มเติมสำหรับการตอบทฤษฎี: ข้อดีของวิธีนี้คือช่วยให้ค้นหาพนักงานในแผนกได้เร็วมากโดยไม่ต้องทำการ Join ข้อมูลระหว่างสองตาราง (No Join).

โจทย์ที่ 8: การจัดการความสัมพันธ์แบบ Many-to-Many (N:M) ทั้ง 3 รูปแบบ

ข้อมูล: ในระบบลงทะเบียนเรียน นักศึกษาหลายคน (Students) ลงทะเบียนเรียนได้หลายรายวิชา (Courses) และได้รับเกรดเฉลี่ย (Enrollments)

- ตาราง Students: S00001 (Willy)
- ตาราง Courses: CRS101 (Database, 3 credits), CRS102 (Web Design, 2 credits)
- ตาราง Enrollments: S00001 ลงทะเบียนวิชา CRS101 ได้เกรด A และลงวิชา CRS102 ได้เกรด B+ ในปี 2026

โจทย์ประยุกต์: จงออกแบบเอกสาร JSON ให้รองรับ Many-to-Many ทั้ง 3 รูปแบบ พร้อมวิเคราะห์กรณีการใช้งาน:

เฉลยแนวทางการตอบและการเขียน JSON:

แบบที่ 1: Embedded Enrollments ไว้ใน Student Collection:

```
{
  "_id": "S00001",
  "studentName": "Willy",
  "enrollments": [
    {
      "courseID": "CRS101",
      "semester": 1,
```

```
"year": 2026,
      "section": 1,
      "grade": "A"
    },
    {
      "courseID": "CRS102",
      "semester": 1,
      "year": 2026,
      "section": 1,
      "grade": "B+"
    }
  ]
}
```

- กรณีใช้งาน: เหมาะมากกับแอปพลิเคชันฝั่งนักศึกษาที่ต้องการ Query ประวัติการเรียนและรายงานเกรดเฉลี่ยรายบุคคล.

แบบที่ 2: Embedded Enrollments ไว้ใน CourseOfferings Collection:

```
{
  "_id": "CRS101",
  "courseName": "Database",
  "credit": 3,
  "semester": 1,
  "year": 2026,
  "section": 1,
  "enrollments": [
    {
      "studentID": "S00001",
      "grade": "A"
    }
  ]
}
```

- กรณีใช้งาน: เหมาะมากกับแอปพลิเคชันระบบสำนักทะเบียนหลังบ้าน ที่ต้องการ Query สรุปยอดจำนวนและผลการเรียนของนักศึกษาที่ลงทะเบียนรายวิชานั้น ๆ.
- ข้อเสีย: หากจะค้นหาว่านักศึกษาคนหนึ่งลงวิชาอะไรไปบ้าง จะค้นหาได้ยากและประสิทธิภาพการทำงานของระบบจะตกลงค่อนข้างมาก.

แบบที่ 3: แยกคอลเลกชัน Enrollments ออกมาต่างหาก (Reference Model):

```
// คอลเลกชัน Enrollments
{
  "courseID": "CRS101",
  "studentID": "S00001",
  "semester": 1,
  "year": 2026,
  "section": 1,
  "grade": "A"
}
```

- กรณีใช้งาน: เหมาะสำหรับความต้องการที่ต้องการ Query ค้นหาข้อมูลได้สะดวกทั้งสองทิศทางอย่างเท่าเทียมกัน.
- ข้อเสีย: ในขั้นตอนประมวลผลดึงรายงานสรุปแบบผสม จะมีความจำเป็นต้องประมวลผลผ่านคำสั่ง \$lookup เพื่อทำคำสั่ง Join ข้อมูลข้ามเอกสาร ซึ่งอาจมีผลกระทบทำให้ประมวลผลได้ช้าลง.

=====

หมวดที่ 4: ตัวอย่างแนวข้อสอบ "ถูก/ผิด (True/False)" และ "เลือกข้อที่ถูกต้อง"
สะสมคะแนนทฤษฎี

จุดประเด็นเหล่านี้ลงในกระดาษโน้ตของคุณเลยครับ!

- ถาม:** หากเอกสาร XML มีการประกาศ XML Declaration ว่า standalone="yes" แต่ในเอกสารมีการเรียกใช้ไฟล์ DTD ภายนอก จะประมวลผลผ่านหรือไม่?
 - ตอบ: ผิด (False).** (หากกำหนดเป็น standalone="yes" ระบบห้ามเรียกใช้ DTD ภายนอกเด็ดขาด มิฉะนั้นตัวแปลภาษา/Parser จะฟ้อง Error ทันที).
- ถาม:** เอกสาร HTML มีความกระชับและมีขีดความสามารถในการจัดเก็บโครงสร้างข้อมูลเพื่อนำไปประมวลผลเชิงลึกได้ดีกว่าเอกสาร XML?
 - ตอบ: ผิด (False).** (HTML ออกแบบมาเฉพาะเพื่อการจัดรูปแบบและนำเสนอหน้าเว็บ/Presentation เท่านั้น ไม่ได้ ออกแบบมาเพื่อแลกเปลี่ยนหรือเก็บข้อมูลสำหรับโปรแกรมประมวลผลอื่น).
- ถาม:** BSON และ Protocol Buffers จัดเป็นข้อมูลมาตรฐานกลางประเภท Binary-based format ซึ่งมนุษย์ไม่สามารถอ่านเข้าใจได้โดยตรง แต่ส่งผ่านเครือข่ายได้รวดเร็วกว่า XML?
 - ตอบ: ถูก (True).** (Binary formats มีขนาดกะทัดรัด บีบอัดข้อมูลเป็นฐานสอง ทำงานรับส่งได้อย่างรวดเร็วและใช้แบนด์วิดท์ต่ำมาก).
- ถาม:** ในระบบ DOM ข้อความที่อยู่ภายในคู่แท็ก เช่น <Name>Miles</Name> ถือเป็นค่า Property ที่เก็บอยู่ในตัว Element Node ชื่อ Name โดยตรง?
 - ตอบ: ผิด (False).** (ข้อความ "Miles" ถือเป็น Text Node ย่อยที่เป็นโหนดลูก/Child Node ซ่อนอยู่ใต้ Element Node ชื่อ Name อีกชั้นหนึ่ง จึงต้องดึงผ่าน firstChild.nodeValue หรือ textContent).
- ถาม:** JSON format รองรับรูปแบบการกำหนด Schema ที่ช่วยตรวจสอบความถูกต้องไวยากรณ์ (Well-formed Verification) ได้หนาแน่นและแข็งแกร่งกว่า XML?
 - ตอบ: ผิด (False).** (XML มี Schema และ DTD ที่แข็งแกร่งในการตรวจสอบความถูกต้องของข้อมูล ซึ่ง JSON ไม่มีเครื่องมือ Schema สนับสนุนในการประเมินโครงสร้างและตรวจสอบโดยตรงเหมือน XML).

=====

สรุปสรุปคำสั่งและโครงสร้างโค้ดที่ออกสอบบ่อยที่สุด (Quick-Reference Manual)

- การแปลงข้อมูลจากโครงสร้างตารางเป็น XML และ JSON
 - JSON Object:** จะต้องเขียนให้อยู่ในรูป "key": "value" pairs ภายในเครื่องหมายปีกกา { } เสมอ โดยที่ **key ต้องถูกหุ้มด้วยเครื่องหมายอัญประกาศคู่ ""** เสมอ ต่างจาก JavaScript Object ที่ไม่จำเป็นต้องมีเครื่องหมายคำพูดหุ้มตัว key (เช่น {name: "Willy Gray"}).
 - Data Types ใน JSON:** รองรับเฉพาะ **string, number** (ไม่ต้องใส่เครื่องหมายคำพูด), **object, array, boolean** (true/false), และ **null**.
 - XML (Extensible Markup Language):** ข้อมูลถูกเก็บเป็น **Tree Structure (โครงสร้างต้นไม้)** และใช้ Elements (แท็ก) ในการอธิบายความหมายข้อมูล ต่างจาก JSON ที่เก็บในลักษณะ **Key-Value Pairs** และรองรับโครงสร้าง Array โดยตรง.

2. คู่มือคำสั่ง JavaScript และ DOM (XML DOM & HTML DOM)

เมื่อเบราว์เซอร์โหลดเอกสาร XML หรือ HTML จะจัดเก็บเอกสารไว้ในหน่วยความจำลักษณะ **Node Tree** ซึ่งเราสามารถใช้ JavaScript เข้าถึงและจัดการข้อมูลผ่าน DOM ได้:

DOM Properties (คุณสมบัติในการเข้าถึงและจัดการสถานะ)

- documentElement** (ความหมาย: เข้าถึง **Root Element** ของเอกสาร XML และ HTML).
 - ตัวอย่างการใช้:** const root = xmlDoc.documentElement;
- childNodes** (ความหมาย: แสดงรายการ **Child Nodes (โหนดลูก)** ทั้งหมดของ Element นั้น ๆ).
 - ตัวอย่างการใช้:** xmlDoc.documentElement.childNodes (ใช้ไปยังลูกลำดับแรก).
- firstChild** (ความหมาย: เข้าถึง **First Child Node (โหนดลูกคนแรก)** ของ Element).
 - ตัวอย่างการใช้:** nodeObj.firstChild.
- nodeValue** (ความหมาย: แสดงหรือแก้ไขค่าข้อความของโหนดนั้น ๆ ซึ่งมักจะชี้ไปที่ **Text Node**).
 - ตัวอย่างการใช้:** nodeObj.firstChild.nodeValue (ดึงค่าข้อความข้างใน).
- textContent** (ความหมาย: ดึงหรือแก้ไขข้อความภายใน Element Node ได้ทันทีโดยไม่ต้องผ่าน Text Node).
 - ตัวอย่างการใช้:** xmlDoc.getElementsByTagName("lastName").textContent.
- style** (ความหมาย: แสดงหรือแก้ไขค่าสไตล์ CSS ของ HTML Element).
 - ตัวอย่างการใช้:** nodeList[x].style.backgroundColor = "red";.

DOM Methods (ฟังก์ชันการสืบค้นและการจัดการโครงสร้าง)

- document.createElement(tag)** (ความหมาย: สร้าง Element Node ใหม่ในหน่วยความจำ).
 - ตัวอย่างการใช้:** const newP = document.createElement("p");
- document.createTextNode(text)** (ความหมาย: สร้าง Text Node (โหนดข้อความตัวอักษร)).
 - ตัวอย่างการใช้:** const txtNode = document.createTextNode("Hello World");
- nodeObj.appendChild(child)** (ความหมาย: เพิ่มโหนดลูก (Child Node) เข้าไปต่อท้ายลำดับสุดท้ายภายในโหนดแม่).
 - ตัวอย่างการใช้:** newP.appendChild(txtNode);
- nodeObj.setAttribute("attribute", "value")** (ความหมาย: แก้ไขหรือกำหนดค่าแอตทริบิวต์ให้กับ Element).
 - ตัวอย่างการใช้:** newLi.setAttribute("class", "list-item");
- document.getElementsByTagName("tag_name")** (ความหมาย: ค้นหา Elements ทั้งหมดที่มีแท็กตรงตามที่กำหนด คืนค่าเป็น Array).
 - ตัวอย่างการใช้:** xmlDoc.getElementsByTagName("firstName") (เอาคนแรกลำดับที่ 2 ของแท็ก firstName).
- DOMParser()** (ความหมาย: ตัวแปลงข้อความธรรมดา (String) ให้กลายเป็น XML Document Object ในหน่วยความจำ).
 - ตัวอย่างการใช้:**

const parser = new DOMParser();

```
const xmlDoc = parser.parseFromString(text, "text/xml"); [8]
```

3. คู่มือการดึงข้อมูลจาก REST API (วิธีเข้าถึงและการแปลงข้อมูล)
ข้อสอบมักเปรียบเทียบการแปลงข้อมูลระหว่างเทคโนโลยีส่งคำขอ 2 แบบหลัก:

- 1. การแปลงแบบ AJAX (XMLHttpRequest):
 - จะรับส่งข้อมูลกลับมาเป็นข้อความดิบ (responseText) และใช้ฟังก์ชัน `JSON.parse()` ในการแปลงข้อความ JSON String เป็น JavaScript Object.
 - โค้ดมาตรฐาน:

```
const obj = JSON.parse(this.responseText); [17]
```

- 1. การแปลงแบบ Fetch API (Promise หรือ Async-Await):
 - ฟังก์ชัน `fetch()` จะมี Method เฉพาะชื่อ `.json()` เพื่อแปลงข้อความ JSON String ให้เป็น JavaScript Object (ซึ่งเมธอดนี้จะคืนค่ากลับมาเป็น Promise เสมอ).
 - ข้อจำกัดห้ามสลับ: `json()` method จะใช้กับ `fetch()` เท่านั้น แต่ `JSON.parse()` จะใช้กับ `callback/AJAX`.
 - โค้ดมาตรฐาน (Promise): `fetch(myapi).then(response => response.json())`.
 - โค้ดมาตรฐาน (Async-Await): `const response = await fetch(url); const json = await response.json();`

4. การแปลงความสัมพันธ์ตาราง (1:M, M:N) เพื่อบันทึกบน MongoDB
ฐานข้อมูล MongoDB จัดเก็บข้อมูลแต่ละเรคคอร์ดในลักษณะ Document (โครงสร้าง JSON) และการยุบโครงสร้างช่วยลดการทำ Join ทำให้สืบค้นข้อมูลได้เร็วขึ้น:

- ➔ ความสัมพันธ์ One-to-Many (1:M)
 - Embedded Document Model (โมเดลฝังตัว - แนะนำ): ยุบโครงสร้างตารางย่อยฝัง Many มาจัดเก็บเป็น Array of Objects ซ้อนอยู่ใน Document หลัก.
 - ข้อดี: เขียน Query ค้นหาข้อมูลได้ง่ายมาก และได้ข้อมูลที่สัมพันธ์กันครบจบในรอบเดียว โดยไม่ต้องมีการเชื่อมโยงหรือ Join ข้อมูลข้ามตาราง ช่วยร่นเวลาค้นหาได้รวดเร็วมาก.
 - โครงสร้าง JSON:

```
{
  "studentID": "S00001",
  "studentName": "Willy",
  "address": [
    { "number": "999-1", "city": "LA", "postcode": 28788 },
    { "number": "555-8", "city": "NY", "postcode": 86777 }
  ]
} [23]
```

- Reference Model (โมเดลอ้างอิง): แยกจัดเก็บคนละคอลเลกชันแล้วเชื่อมโยงผ่าน ID (เช่น คอลเลกชัน Students และคอลเลกชัน Addresses แยกกัน).
 - ข้อเสีย: การ Query สลับไปมาต้องใช้คำสั่งที่มีความซับซ้อนและจำเป็นต้องพึ่งพา `$lookup` ในการ JOIN เชื่อมต่อข้อมูลระหว่างคอลเลกชัน.

- 5. คู่มือคำสั่งใน MongoDB Shell (การจัดการข้อมูลและ Aggregation)
 - `$pull` (ความหมาย: ดึงหรือลบสมาชิกตัวใดตัวหนึ่งออกจากฟิลด์ประเภท Array).

- *ไวยากรณ์:* `db.books.updateOne({ "title": "What a bee knows" }, { $pull: { "genres": "Science" } })`.
- `$push` (ความหมาย: เพิ่มสมาชิกตัวใหม่เข้าไปจัดเก็บในฟิลด์ประเภท Array).
 - *ไวยากรณ์:* `db.books.updateOne({ "title": "What a bee knows" }, { $push: { "genres": "Science" } })`.
- `$each` ร่วมกับ `$push` (ความหมาย: เพิ่มข้อมูลพร้อมกันหลายค่าลงไป ในฟิลด์ Array).
 - *ไวยากรณ์:* `db.books.updateMany({ "author": "Mark Waid" }, { $push: { "genres": { $each: ["Cartoon", "Funny"] } } })`.
- `$in` ร่วมกับ `$pull` (ความหมาย: คัดเลือกและลบข้อมูลพร้อมกันหลายค่าออกจากฟิลด์ Array).
 - *ไวยากรณ์:* `db.books.updateMany({ "author": "Mark Waid" }, { $pull: { "genres": { $in: ["Cartoon", "Funny"] } } })`.
- `$unwind` (ความหมาย: คลี่คลายโครงสร้าง Array (Deconstruct) ออกมาเพื่อสร้างเอกสารย่อยแยกเฉพาะเจาะจงทีละหนึ่งรายการ).
 - *ตัวอย่างการใช้:* `{ $unwind: "$items" }`.
- `$lookup` (ความหมาย: ใช้รวมหรือเชื่อมโยงข้อมูลระหว่างสองคอลเลกชันที่สัมพันธ์กัน (เสมือนการ Join ในระบบ RDB)).
 - *ไวยากรณ์:*

```
{
  $lookup: {
    from: "publishers",           // คอลเลกชันที่จะดึงมาเชื่อม [35]
    localField: "pubid",         // ฟิลด์เชื่อมโยงในคอลเลกชันหลัก [35]
    foreignField: "publisherid", // ฟิลด์เชื่อมโยงของคอลเลกชันปลายทาง [35]
    as: "publisher_details"      // ชื่อฟิลด์ผลลัพธ์ที่จะเก็บก่อน Object ที่ได้
  }
}
```

=====

Advanced Table-to-Serialization & Code Master Guide
(คู่มือวิเคราะห์โครงสร้างตารางยากพิเศษ และโปยเติมโค้ดขั้นสูงสำหรับสอบ Lecture และ Lab)

สมรภูมิที่ 1: การแปลงตารางเชิงสัมพันธ์ (RDB) เป็น XML และ JSON
โจทย์ประยุกต์มักกำหนดตารางเชิงสัมพันธ์ที่มีความเชื่อมโยงกันมาให้ แล้วสั่งให้แปลงข้อมูลให้อยู่ในรูปแบบ XML และ JSON ที่ถูกต้องตามหลักไวยากรณ์

1.1 ตารางโจทย์ตัวอย่าง (ระดับซับซ้อน): ระบบสั่งซื้อสินค้า (Order System)

ตารางที่ 1: CUSTOMERS (ข้อมูลลูกค้า) | CustomerID | CustName | Email | | :-- | :-- | :-- | | C001 | Robert Lee | robert@mail.com | | C002 | Mary Jane | mary@mail.com |

ตารางที่ 2: ORDERS (ข้อมูลใบสั่งซื้อ - ความสัมพันธ์ 1:M กับ Customers) | OrderID | OrderDate | TotalPrice | CustomerID | | :-- | :-- | :-- | :-- | | ORD-99 | 28/08/2026 | 1500 | C001 | | ORD-10 | 29/08/2026 | 3200 | C001 |

ตารางที่ 3: ORDER_ITEMS (รายการสินค้าในออเดอร์ - ความสัมพันธ์ M:N ระหว่าง Orders & Products) | OrderID | ProductID | Qty | PricePerUnit | | :-- | :-- | :-- | :-- | | ORD-99 | P001 | 2 | 500 | | ORD-99 | P002 | 1 | 500 | | ORD-10 | P001 | 4 | 500 |

1.2 การแปลงเป็น XML (Element-Based vs. Attribute-Based)

การทำข้อมูลให้เป็น Well-Formed XML ต้องคำนึงถึงกฎหลัก:

- 1. ต้องมี **Root Element** เพียง **1 ตัว** คลุมข้อมูลทั้งหมดไว้.
- 2. แท็กเปิดและแท็กปิดต้องเขียนเหมือนกัน สะกดตรงกัน และปิดเป็นลำดับชั้น (Case-Sensitive, No Overlapping).
- 3. ข้อมูลที่เป็นช่องว่างหรือไม่มีค่า สามารถเขียนเป็น **Empty Element** เช่น <Publisher/> หรือ <Publisher></Publisher>.

รูปแบบที่ 1: Element-Based XML (เน้นการใช้แท็กย่อยเก็บข้อมูลทั้งหมด)
เหมาะสำหรับการแสดงผลข้อมูล ข้อความยาวๆ หรือข้อมูลที่มีโครงสร้างซับซ้อน.

<?xml version="1.0" encoding="UTF-8"?>

<OrderSystem>

<Customers>

<Customer>

<CustomerID>C001</CustomerID>

<CustName>Robert Lee</CustName>

<Email>robert@mail.com</Email>

<Orders>

<Order>

<OrderID>ORD-99</OrderID>

<OrderDate>28/08/2026</OrderDate>

<TotalPrice>1500</TotalPrice>

<OrderItems>

<OrderItem>

<ProductID>P001</ProductID>

<Qty>2</Qty>

<PricePerUnit>500</PricePerUnit>

</OrderItem>

<OrderItem>

<ProductID>P002</ProductID>

<Qty>1</Qty>

<PricePerUnit>500</PricePerUnit>

</OrderItem>

</OrderItems>

</Order>

</Orders>

</Customer>

</Customers>

</OrderSystem>

รูปแบบที่ 2: Attribute-Based XML (ใช้แอตทริบิวต์เก็บ Metadata หรือ ID)
แอตทริบิวต์ (Attribute) จะอยู่ในแท็กเปิด มีลักษณะเป็น name="value" เหมาะกับการเก็บข้อมูลสั้นๆ ที่ใช้ในการประมวลผล เช่น Key หลักหรือ ID และลำดับของ Attribute จะไม่มีความสำคัญ ต่างจาก Element ที่ลำดับมีความสำคัญ.

<?xml version="1.0" encoding="UTF-8"?>

<OrderSystem>

<Customers>

<Customer id="C001" email="robert@mail.com">

<CustName>Robert Lee</CustName>

<Orders>

<Order id="ORD-99" date="28/08/2026" total="1500">

<OrderItems>

<OrderItem prodId="P001" qty="2" price="500" />

<OrderItem prodId="P002" qty="1" price="500" />

</OrderItems>

</Order>

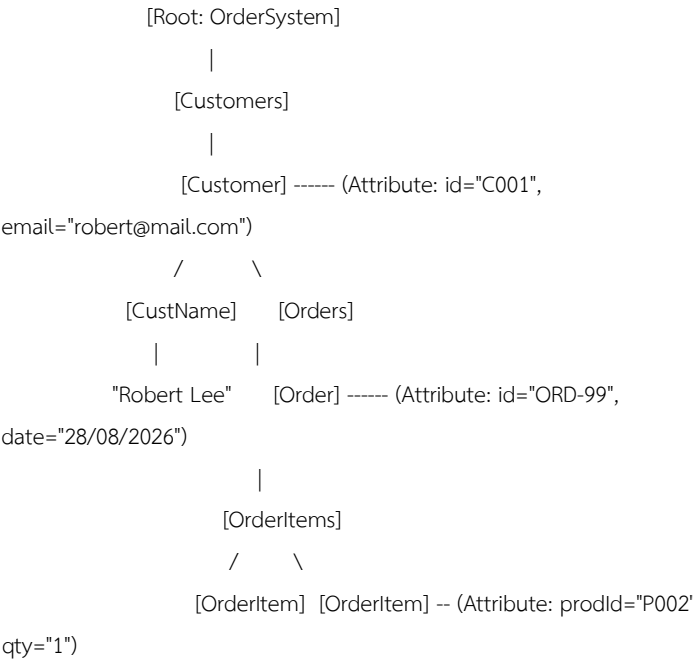
</Orders>

</Customer>

</Customers>

</OrderSystem>

รูปแบบที่ 3: XML Tree Structure (โครงสร้างต้นไม้สำหรับใช้วาดในข้อสอบ)
หากข้อสอบสั่งให้วาดโครงสร้าง Tree Structure สอดคล้องกับเอกสาร XML ให้วาดแตกกิ่งก้านสาขาโดยยึดโครงสร้างลำดับชั้นจากบนลงล่างดังนี้:



1.3 การแปลงเป็น JSON (Array of Objects)

ไวยากรณ์หลักของ JSON ที่ต่างจาก JavaScript Object คือ **"Key"** ใน JSON ทุกตัวจำเป็นต้องครอบด้วยเครื่องหมายอัญประกาศคู่ "" เสมอ. ข้อมูลสามารถเก็บประเภทตัวเลข (Number), ข้อความ (String), ค่าความจริง (Boolean), อาร์เรย์ (Array) หรือข้อมูลว่าง (null) ได้.

```
[
  {
    "customerId": "C001",
    "custName": "Robert Lee",
    "email": "robert@mail.com",
    "orders": [
      {
        "orderId": "ORD-99",
        "orderDate": "28/08/2026",
        "totalPrice": 1500,
        "orderItems": [
          { "productId": "P001", "qty": 2, "pricePerUnit": 500 },
          { "productId": "P002", "qty": 1, "pricePerUnit": 500 }
        ]
      }
    ]
  },
]
```

```
{
  "orderId": "ORD-10",
  "orderDate": "29/08/2026",
  "totalPrice": 3200,
  "orderItems": [
    { "productId": "P001", "qty": 4, "pricePerUnit": 500 }
  ]
}
```

สมรภูมิที่ 2: การออกแบบฐานข้อมูล NoSQL (MongoDB Schema Design) เมื่อต้องนำโครงสร้าง RDB ข้างต้นไปออกแบบใน MongoDB โครงสร้างข้อมูลจะเก็บเป็น JSON Document ซึ่งเปิดโอกาสให้ออกแบบได้หลากหลายรูปแบบตามประเภทการค้นหา.

2.1 การจัดการความสัมพันธ์แบบ 1-M (Customer และ Orders) ใน MongoDB สามารถออกแบบความสัมพันธ์แบบ 1-M ได้ 2 รูปแบบใหญ่ๆ: รูปแบบที่ 1: Embedded Document Model (การฝังข้อมูลอาร์เรย์ไว้ข้างใน) เป็นการนำข้อมูลฝั่ง Many (Orders) ฝังเป็นอาร์เรย์ของ Object ไว้ในฝั่ง One (Customer):

```
// Customers Collection
{
  "_id": ObjectId("65840000000000000000000000000001"),
  "customerId": "C001",
  "custName": "Robert Lee",
  "email": "robert@mail.com",
  "orders": [
    { "orderId": "ORD-99", "orderDate": "28/08/2026", "totalPrice": 1500 },
    { "orderId": "ORD-10", "orderDate": "29/08/2026", "totalPrice": 3200 }
  ]
}
```

- ข้อดี: ดึงข้อมูลลูกค้าพร้อมรายละเอียดการสั่งซื้อทั้งหมดได้ใน Query เดียว รวดเร็วมาก ไม่ต้องใช้กระบวนการ Join.
- ข้อเสีย: หากมีการสั่งซื้อเกิดขึ้นจำนวนมาก ขนาดของ Document อาจโตเกินลิมิต 16MB ของ MongoDB และยากหากต้องการ Query ดูรายงานการสั่งซื้อแยกอิสระจากรายชื่อลูกค้า.

รูปแบบที่ 2: Reference Document Model (การเก็บแบบอ้างอิง ID) แยก Collection ออกจากกันอย่างอิสระ แล้วใช้ ID อ้างอิงระหว่างกัน:

```
// Customers Collection
{
  "_id": ObjectId("65840000000000000000000000000001"),
  "customerId": "C001",
  "custName": "Robert Lee",
  "email": "robert@mail.com"
}
```

```
// Orders Collection
{
  "_id": ObjectId("65841000000000000000000000000099"),
  "orderId": "ORD-99",
```

```
"orderDate": "28/08/2026",
"totalPrice": 1500,
"customerId": "C001"
}
```

- ข้อดี: โครงสร้างยืดหยุ่น ข้อมูลในแต่ละ Collection ไม่ซ้ำซ้อนและไม่มีปัญหาเรื่องขนาดเอกสารเกินขีดจำกัด.
- ข้อเสีย: การค้นหาข้อมูลแบบเชื่อมโยงกันจำเป็นต้องทำ Left Outer Join ผ่านคำสั่ง \$lookup ซึ่งเพิ่มขั้นตอนและส่งผลกระทบต่อประสิทธิภาพความเร็ว.

2.2 การจัดการความสัมพันธ์แบบ M-N (Orders และ Products) อ้างอิงตามทฤษฎีในบทเรียน (ที่ยกตัวอย่างวิชาเรียนและการลงทะเบียนเรียน) ความสัมพันธ์แบบ Many-to-Many สามารถออกแบบได้ 3 แบบหลัก ดังนี้: แบบที่ 1: Embedded ในฝั่งหนึ่ง เช่น ฝั่งรายการสินค้า (OrderItems) ไว้ใน Orders Collection

เหมาะกับกรณีที่เราเน้นสืบค้นทางฝั่ง Orders เป็นหลัก เช่น การออกใบเสร็จหรือตรวจสอบรายการสั่งซื้อ:

```
// Orders Collection
{
  "_id": ObjectId("65841000000000000000000000000099"),
  "orderId": "ORD-99",
  "orderDate": "28/08/2026",
  "totalPrice": 1500,
  "items": [
    { "productId": "P001", "qty": 2, "pricePerUnit": 500 },
    { "productId": "P002", "qty": 1, "pricePerUnit": 500 }
  ]
}
```

- ข้อดี: ดึงออเดอร์ออกมาจะเห็นรายการและข้อมูลสินค้าที่ซื้อทันที (ประสิทธิภาพดีมากเมื่อพิมพ์ใบเสร็จ).
- ข้อเสีย: ถ้าจำเป็นต้องแก้ไขรายละเอียดของสินค้า เช่น เปลี่ยนชื่อหมวดหมู่ของสินค้า P001 จะต้องไล่ตามแก้ไขในทุกๆ ออร์เดอร์ที่มีการสั่งซื้อสินค้านี้.

แบบที่ 2: Embedded ในอีกฝั่งหนึ่ง เช่น ฝั่งประวัติการซื้อในฝั่ง Products/Store Collection

เหมาะสำหรับการวิเคราะห์ดูว่าสินค้าตัวนี้มีใครมาซื้อไปบ้าง.

แบบที่ 3: แยก Collection กลางเป็นประวัติ (OrderItems / Enrollments Collection) แล้วเก็บ Reference เป็นรูปแบบที่ยืดหยุ่นที่สุด สามารถ Query เชื่อมโยงได้สองทิศทาง แต่ต้องแลกกับการประมวลผลด้วย Aggregate Pipeline ขั้นสูง:

```
// OrderItems Collection (ตารางกลางเก็บข้อมูลความสัมพันธ์)
{
  "_id": ObjectId("65842000000000000000000000000001"),
  "orderId": "ORD-99",
  "productId": "P001",
  "qty": 2,
  "pricePerUnit": 500
}
```

2.3 โฟกัสเขียนคำสั่ง Aggregation Pipeline ระดับยาก (พร้อมออกปฏิบัติ Lab) กรณีเลือกการออกแบบความสัมพันธ์รูปแบบที่ 3 (แยก Collection) แล้วโจทย์ต้องการให้เรา Query ดึงข้อมูลออเดอร์ โดยต้องการนำรายละเอียดข้อมูลของสินค้า

จาก Products Collection มารวมอยู่ข้างในออร์เดอร์ด้วย ต้องเขียนขั้นตอนการสืบค้นดังนี้:

```
// ค้นหาจาก collection "order_items" เพื่อ Join ดึงชื่อสินค้าจาก "products"
db.order_items.aggregate([
  // ขั้นที่ 1: ทำการ Join (Left Outer Join) ข้อมูลจาก products collection
  {
    $lookup: {
      from: "products",          // collection ปลายทางที่ต้องการ Join
      localField: "productId",   // field ใน collection ปัจจุบัน (order_items)
      foreignField: "productId", // field ใน collection ปลายทาง (products)
      as: "product_details"      // ชื่อ field อาร์เรย์เก็บผลลัพธ์การ Join
    }
  },
  // ขั้นที่ 2: คลี่ข้อมูลอาร์เรย์ (product_details) ให้แผ่ออกมาเป็น Object เดี่ยว
  {
    $unwind: "$product_details"
  },
  // ขั้นที่ 3: เลือกฟิลด์ที่ต้องการจัดรูปแบบการแสดงผล [27]
  {
    $project: {
      _id: 0,
      orderId: 1,
      productId: 1,
      quantity: "$qty",
      productName: "$product_details.Title",
      category: "$product_details.Category",
      subTotal: { $multiply: ["$qty", "$pricePerUnit"] } // สูตรคำนวณราคารวม
    }
  }
])
```

สมรภูมิที่ 3: โฟयरการเล่นและเติมโค้ด JavaScript (DOM Parser, DOM Navigation & REST API)

นี่คือส่วนที่ออกสอบแบบเติมคำในช่องว่างบ้อยที่สุดของ Lecture โดยเน้นการเข้าถึง โหนดและจัดการโครงสร้าง HTML/XML ผ่าน JavaScript.

3.1 กับดัก "Text Node" ใน DOM Navigation (ห้ามลืมเด็ดขาด!)

เมื่อบรรดาเซอร์ทำการโหลด XML หรือ HTML เข้ามาสู่หน่วยความจำ ตัวเอกสารจะถูกจัดเก็บเป็น Object ต้นไม้ที่ประกอบไปด้วยกิ่งก้านของ Node ต่างๆ.

- **กฎเหล็ก:** ข้อความสัญญาณใดๆ ที่อยู่ภายใต้ Element ของ XML/HTML จะถือว่าเป็น **Text Node** แยกต่างหาก.
- ดังนั้น หากคุณต้องการดึงคำว่า "Alan" จากแท็ก `<firstName>Alan</firstName>`:
 - ❌ เขียนโค้ดเข้าถึง Element: `xmlDoc.getElementsByTagName("firstName")`
 - **สิ่งที่你会ได้:** คุณจะได้ Element Node ของแท็ก `<firstName>` ไม่ใช่ข้อความข้างใน!
 - ✅ วิธีการสืบค้นเพื่อหาค่าจริง:
 - `xmlDoc.getElementsByTagName("firstName").firstChild.nodeValue` (ดึงลูกตัวแรกที่ เป็น Text Node แล้วเอาค่า)

- `xmlDoc.getElementsByTagName("firstName").childNodes.nodeValue` (เหมือนข้างบน)
- `xmlDoc.getElementsByTagName("firstName").textContent` (วิธีลัดที่สะดวกและปลอดภัยที่สุด).

3.2 เทมเพลตเติมโค้ดแปลง XML String ด้วย DOMParser

ในข้อสอบมักจะให้ XML String มาตัวหนึ่งแล้วสั่งให้ใช้ JavaScript แปลงเป็น DOM Document Object และเข้าถึงข้อมูล:

```
<script>
  // ข้อมูล XML ที่รับมาในรูปแบบ String
  var xmlString = "<persons>"+
    "  <person pid='p01'>"+
    "    <firstName>Alan</firstName>"+
    "    <lastName>Wilkes</lastName>"+
    "  </person>"+
    "</persons>";

  // 1. วิดีเติมโค้ดเพื่อสร้าง DOMParser Object [33]:
  var parser = new DOMParser();

  // 2. วิดีเติมโค้ดเพื่อ Parse String ให้กลายเป็น XML Document Object [33]:
  var xmlDoc = parser.parseFromString(xmlString, "text/xml");

  // 3. ตัวอย่างคำสั่งสืบค้นเข้าถึงข้อมูลที่ออกสอบบ่อย [30, 31]:

  // ก) เข้าถึงชื่อโหนด Root -> จะพิมพ์ค่า "persons" ออกมา [33]
  console.log(xmlDoc.documentElement.nodeName);

  // ข) ดึงค่า "Alan" โดยใช้ Property อย่างเดียว (เดินตามพิกัด Tree) [30, 31]
  // ลำดับ: root -> childNodes (person) -> childNodes (firstName) ->
  firstChild (Text Node) -> nodeValue
  var firstNameVal =
xmlDoc.documentElement.childNodes.childNodes.firstChild.nodeValue;
  console.log(firstNameVal); // ผลลัพธ์: Alan

  // ค) ดึงค่า "Wilkes" โดยใช้ Method ร่วมกับ Property [31, 33]
  var lastNameVal =
xmlDoc.getElementsByTagName("lastName").firstChild.nodeValue;
  console.log(lastNameVal); // ผลลัพธ์: Wilkes

  // ง) การดึงค่าข้อมูลแอตทริบิวต์ (Attribute value) "p01" ของคนแรก [34]
  var pidAttr =
xmlDoc.getElementsByTagName("person").getAttribute("pid");
  console.log(pidAttr); // ผลลัพธ์: p01
</script>
```

3.3 คลังโค้ดคำสั่งสำหรับการสร้างและแก้ไข HTML DOM Node

ข้อสอบ Lab มักให้เขียนฟังก์ชันสำหรับการเพิ่มคลาส สร้าง Element ตกแต่งสไตล์ หรือลบโหนดข้อมูลทิ้ง.

// กำหนดโหนดพ่อที่ต้องการจะนำของใหม่ไปใส่ [35]

```
const parentDiv = document.getElementById("div1");
```

```

// 1. การสร้าง Element Node ใหม่ขึ้นมา [36, 37]
const newPara = document.createElement("p");

// 2. การสร้าง Text Node เพื่อเก็บข้อความ [35]
const newText = document.createTextNode("นี่คือย่อหน้าที่เพิ่มเข้ามาใหม่");

// 3. นำ Text Node ไปใส่ใน Element Node ใหม่ [35]
newPara.appendChild(newText);

// 4. การเพิ่มคุณลักษณะ (Attribute) เช่น กำหนด id ให้กับโหนดใหม่ [38, 39]
newPara.setAttribute("id", "p3"); // หรือจะเขียนสั้นๆ: newPara.id = "p3";

// 5. การกำหนด CSS Styles ผ่านสไตล์ชีตโดยตรง [40, 41]
newPara.style.backgroundColor = "yellow";
newPara.style.fontWeight = "bold";

// 6. นำ Element ใหม่ที่ประกอบร่างเสร็จแล้ว ไปต่อท้ายใน Div พ่อ [35]
parentDiv.appendChild(newPara);

// 7. การลบ Element ตัวเก่าออกจากการแสดงผลหน้าเว็บ [42]
document.getElementById("p1").remove();

```

3.4 โพลีเทมโค้ดสืบทอด REST API (XHR/AJAX vs. Fetch vs. Async-Await)

นี่คือหัวข้อสำคัญที่สุดในการเชื่อมต่อรับส่งข้อมูลระหว่าง Client และ Server โดย Server มักจะคืนข้อมูลในรูปแบบของ JSON เพื่อให้ให้นำข้อมูลนี้ไปจัดการบนคอมพิวเตอร์ Client ต่อได้โดยตรงเลย (ไม่ใช่ HTML).

```

const apiUrl = "https://dummyjson.com/users/1";

// -----
// แบบที่ 1: AJAX (XMLHttpRequest) แบบดั้งเดิม [42]
// -----

function loadDataWithAJAX() {
  // สร้างอ็อบเจกต์เชื่อมต่อ
  var xhr = new XMLHttpRequest();

  // ผู้การทำงานเมื่อมีการเปลี่ยนสถานะการเชื่อมต่อ
  xhr.onreadystatechange = function() {
    // กฎเหล็กตรวจสอบ: ตรวจสอบความสำเร็จสมบูรณ์ (readyState == 4 และ
    status == 200) [42]
    if (this.readyState == 4 && this.status == 200) {

      // แปลงข้อมูลที่ได้ซึ่งเป็น JSON String ให้เป็น JavaScript Object เสมอ [44-46]
      var userObj = JSON.parse(this.responseText);

      // พิมพ์ค่าเพื่อตรวจสอบความถูกต้อง
      console.log(userObj.firstName + " " + userObj.lastName);
      document.getElementById("fullname").textContent =
userObj.firstName;
    }
  }
}

```

```

};

// เปิดการเชื่อมต่อส่งคำขอแบบ GET [44]
xhr.open("GET", apiUrl, true);
xhr.send();
}

// -----
// แบบที่ 2: Fetch API (Promise-based) [47]
// -----

function loadDataWithFetch() {
  fetch(apiUrl)
    // ขั้นตอนที่ 1: ต้องทำการคืนค่าผลการแปลง stream ของข้อมูลให้อยู่ในรูปแบบ
JSON Object เสมอ [47]
    .then(function(response) {
      return response.json();
    })
    // ขั้นตอนที่ 2: ดึงอ็อบเจกต์ที่แปลงแล้วไปใช้งาน [47]
    .then(function(myJsonObj) {
      console.log(myJsonObj.firstName);
      document.querySelector(".fullname").textContent =
myJsonObj.firstName;
    })
    // ตรวจสอบกรณีการเชื่อมต่อขัดข้อง
    .catch(function(error) {
      console.error("เกิดข้อผิดพลาด:", error);
    });
}

// -----
// แบบที่ 3: Async/Await (รูปแบบสมัยใหม่ที่อ่านและเขียนง่ายที่สุด) [48]
// -----

async function loadDataWithAsyncAwait() {
  try {
    // ส่ง Request และรอผลตอบกลับ (Await) [48]
    const response = await fetch(apiUrl);

    // แปลงผลตอบกลับเป็น JSON และรอผลลัพธ์สำเร็จ [48]
    const userObj = await response.json();

    // นำไปพ่นลงในโครงสร้าง HTML DOM ทันที [48]
    document.querySelector('.fullname').textContent = userObj.firstName
+ " " + userObj.lastName;
    document.querySelector('.email').textContent = userObj.email;
    document.querySelector('.avatar').src = userObj.image; // แก้ไข URL
รูปภาพ [48, 49]

  } catch (error) {
    console.error("ผิดพลาดขณะดึง REST API:", error);
  }
}

```

ตารางเปรียบเทียบการแปลงข้อมูลระหว่าง AJAX และ Fetch API
ใช้แยกแยะเพื่อตอบพาร์ทเขียนโค้ด | จุดเปรียบเทียบ | AJAX (XMLHttpRequest) | Fetch API / Async-Await | | :--- | :--- | :--- | | ชนิดข้อมูลตั้งต้น | this.responseText (ข้อมูลเป็น JSON Stringดิบ). | response (ข้อมูลเป็น Response Stream). | | คำสั่งแปลงข้อมูล | JSON.parse(this.responseText) (เป็น synchronous). | response.json() (เป็น asynchronous คืนค่าในรูป Promise). | | ผลลัพธ์หลังแปลง | JavaScript Object ที่เข้าถึงคุณสมบัติได้ทันที. | JavaScript Object ที่ส่งผ่านเข้าไปยัง .then() ถัดไป. |

DATA SERIALIZATION LECTURE EXAM COMPLETE COMPENDIUM

CHAPTER 1: INTRODUCTION TO DATA SERIALIZATION

1. นิยามพื้นฐาน (Core Definitions)

- **Data Serialization** คือ กระบวนการแปลงข้อมูลจาก Object ที่อยู่ในโครงสร้างซับซ้อนในหน่วยความจำของเครื่องต้นทาง ให้กลายเป็นข้อมูลในรูปแบบมาตรฐานกลาง (Data Serialization Formats) ทั้งแบบ Text หรือ Binary เพื่อให้เหมาะสมกับการส่งผ่านเครือข่ายหรือการจัดเก็บลงอุปกรณ์คอมพิวเตอร์.
- **Data Deserialization** คือ กระบวนการย้อนกลับ โดยแปลงข้อมูลมาตรฐานกลางที่รับมา ให้กลับไปเป็น Object ในหน่วยความจำปลายทาง ซึ่งจะมีโครงสร้างและข้อมูลเหมือนกับ Object ดั้งเดิมบนเครื่องต้นทางทุกประการ.
- **ปัญหาหลักของการแลกเปลี่ยนข้อมูล:** ระบบคอมพิวเตอร์และแพลตฟอร์มต่าง ๆ มีความแตกต่างกันในการออกแบบโครงสร้าง ข้อมูลในหน่วยความจำจึงจัดเก็บต่างกัน ทำให้ไม่สามารถส่ง Object ข้ามระบบไปใช้งานตรง ๆ ได้โดยไม่มีตัวกลาง.

2. การประยุกต์ใช้งาน (Applications)

1. **การจัดเก็บข้อมูลในฐานข้อมูล (Storing data into Databases):** เช่น การแปลง Java Object ให้เป็น Byte Streams แล้วจัดเก็บในฐานข้อมูลเชิงสัมพันธ์ (เช่น MySQL) ผ่านเทคโนโลยี Java JDBC.
2. **การรับส่งข้อมูลผ่านเครือข่าย (Transfer data to the network):** เช่น การส่งข้อมูล Object ระหว่าง Client และ Server ใน Web Applications และ Mobile Applications.
3. **การใช้ข้อมูลร่วมกันในระบบกระจายตัว (Sharing data in a Distributed Object Model):** แลกเปลี่ยนข้อมูลระหว่างเครื่องคอมพิวเตอร์ที่มีแพลตฟอร์มแตกต่างกัน ผ่านเทคโนโลยี Web Services เช่น REST หรือ SOAP.
4. **การเรียกใช้เมธอดระยะไกล (Remote Method Invocation - RMI):** การส่ง Serialized Object ในลักษณะของ Parameter ไปยังฟังก์ชันที่ทำงานอยู่บนเครื่องอื่นในเครือข่าย (Remote Machine).

3. ข้อจำกัดของ HTML และการเกิด Dynamic Web Page

- **ข้อเสียของ HTML:** ออกแบบมาเพื่อกำหนดการจัดวางและการนำเสนอข้อมูล (Presentation) บนบราวเซอร์เท่านั้น ไม่เหมาะกับการส่งข้อมูลดิบเพื่อนำไปประมวลผลต่อ และเป็นแบบ Read-only หรือ **Static Web page**.
- **Dynamic Web Page:** คือหน้าเว็บที่ Client สามารถนำข้อมูลดิบจาก Server ไปประมวลผล หรือแสดงผลในรูปแบบใดก็ได้ตามต้องการอย่างอิสระ หรือเปิดโอกาสให้ผู้ใช้งานสร้างเนื้อหา (Content) ขึ้นบนเว็บได้ด้วยตนเอง.
- **ความจำเป็นของ Data Serialization:** เพื่อพัฒนา Dynamic Web Page จึงต้องสร้างภาษากลางที่เป็นมาตรฐานขึ้นมาแทน HTML เพื่อแปลง

Java Object ฝั่ง Server ให้เป็นภาษากลางที่ Client แพลตฟอร์มอื่น ๆ เข้าใจได้.

4. การแลกเปลี่ยนข้อมูลบน Mobile Devices

- สมาร์ทโฟนต่างค่ายต่างระบบปฏิบัติการ (iOS ของ Apple และ Android ของ Google) มีเทคโนโลยีพัฒนาแอปพลิเคชันต่างกัน แต่แอปพลิเคชันทั่วไป เช่น SMS, Calendar, Maps พัฒนาด้วยภาษา Java ได้.
- การใช้ Data Serialization ช่วยให้การรับส่งข้อมูลระหว่างแอปพลิเคชันในระบบเครือข่ายมี **ขนาดข้อมูลที่เล็กและส่งได้อย่างรวดเร็ว**.

5. การแบ่งกลุ่ม Data Serialization Formats

1. **Text-Based Formats** (มนุษย์สามารถอ่านเข้าใจได้): เช่น XML, JSON, YAML.
 - **ข้อดี:** สามารถถูกเขียนและทำ Deserialization ได้บนระบบที่หลากหลาย แนะนำสำหรับการแลกเปลี่ยนข้อมูลผ่าน Web Services ทั่วไป.
2. **Binary-Based Formats** (มนุษย์ไม่สามารถอ่านเข้าใจได้): เช่น BSON (Binary JSON), MessagePack, Protocol Buffers (Protocol Buffers), Apache Thrift.
 - **ข้อดี:** มีขนาดไฟล์ที่เล็กกว่ามาก และมีความเร็วในการรับส่งมากกว่ากลุ่ม Text-Based.

6. ไทม์ไลน์ปี ค.ศ. และ Milestones ที่สำคัญ

- **ค.ศ. 2001:** Douglas Crockford พัฒนาภาษา JSON เป็นครั้งแรก.
- **ค.ศ. 2004:** ภาษา YAML เวอร์ชันแรกเกิดขึ้น โดยพัฒนาเป็น Extension ของ JSON เพื่อให้โครงสร้างง่ายขึ้น.
- **ค.ศ. 2008:** Google ปลอ่ยตัว Protocol Buffers (ProtoBuf) ออกมาอย่างเป็นทางการ.
- **ค.ศ. 2009:** BSON ถูกปลอ่ยออกมาเพื่อใช้เป็นฟอร์แมตเก็บข้อมูลภายในของฐานข้อมูล MongoDB.
- **ค.ศ. 2017:** ภาษา JSON ได้รับการประกาศยกขึ้นเป็นมาตรฐานและใช้งานอย่างเป็นทางการ.

CHAPTER 2.1 & 2.2: XML & DOCUMENT OBJECT MODEL (DOM)

1. นิยามและการประกาศโครงสร้าง XML

- **XML (Extensible Markup Language):** เป็น Subset ของภาษา SGML (ปี 1980) ได้รับการพัฒนาโดย W3C เพื่อให้ง่ายต่อการแลกเปลี่ยนข้อมูลในระบบอินเทอร์เน็ต.
- **XML Declaration:** เป็นคำสั่งประกาศโครงสร้างบรรทัดแรกของเอกสาร.
 - **รูปแบบ:** `<?xml version="version_num" encoding="encoding_type" standalone="yes|no" ?>`.
 - **ความหมายของ standalone:**
 - **standalone="yes"** หมายถึง เอกสารนี้ทำงานเดี่ยว ๆ ไม่มีการเรียกใช้เอกสารภายนอก เช่น DTD เพิ่มเติม หากตั้งค่านี้นี้แต่ไปเรียกใช้ DTD ตัว XML Parser จะแจ้ง Error ทันที.
 - **standalone="no"** (ค่า Default) ยอมให้เรียกใช้งานร่วมกับ DTD หรือ Schema ภายนอกได้.

2. กฎหลักของ Well-Formed XML Document (ออกสอบบ่อยมาก)

หากเอกสารไม่เป็นไปตามกฎเหล่านี้ จะถือว่าไม่ใช่ Well-Formed และจะไม่สามารถ Parse ผ่านได้:

1. **ชื่อ Element (Tag Name)** ต้องเป็นไปตามข้อกำหนดของ W3C:
 - **ต้องขึ้นต้นด้วย** ตัวอักษร (Letter), Underscore (_), หรือ Colon (:) เท่านั้น.

- ห้ามขึ้นต้นด้วย ตัวเลข หรือสัญลักษณ์พิเศษอื่น ๆ.
- ห้ามมีช่องว่าง (Blank Space) หรือสัญลักษณ์พิเศษ เช่น *, # ปรากฏในชื่อเด็ดขาด.
- ตัวอย่างชื่อที่ผิด (Invalid): <1Name> (ขึ้นต้นด้วยตัวเลข), <first name> (มีช่องว่าง), <Book#category> (มีเครื่องหมายพิเศษ).

2. Tag เปิดและ Tag ปิดต้องตรงกัน: มีการเปิด-ปิดเป็นคู่ และระบบ XML เป็นแบบ Case-Sensitive (เช่น <CD>...</cd> ถือว่าผิดกฎ).
3. ห้ามเกิดการคร่อมแท็ก (No Overlapping): หากเปิด Element ด้านใน ต้องปิด Element ด้านในให้เสร็จสิ้นก่อนปิดแท็กด้านนอก.
 - ผิด (Overlapped): <Book><Name>XML Technology</Book></Name>.
 - ถูกต้อง: <Book><Name>XML Technology</Name></Book>.
4. ต้องมี Root Element เพียง 1 แท็กเท่านั้น ที่ครอบคลุม Element ย่อย ทั้งหมดภายในหน้าเอกสาร.
5. Empty Element: Element ว่างที่ไม่มีคำข้อมูลภายใน สามารถเขียนได้ 2 แบบคือ <student/> หรือ <student></student>.

3. ข้อแตกต่างเชิงเปรียบเทียบ: Element vs. Attribute

คุณสมบัติ	Element (Metadata Tag)	Attribute (Name-Value Pair)
โครงสร้างภายใน	สามารถมี Sub-elements (แท็ก ไม่สามารถมี Sub-attributes ได้ ย่อย) ซ้อนข้างในได้	
ขนาดข้อมูล	เหมาะสำหรับการจัดเก็บข้อมูล	นิยมใช้จัดเก็บเฉพาะข้อความสั้น ข้อความยาว ๆ
วัตถุประสงค์หลัก	ใช้เก็บข้อมูลสำหรับการแสดงผล (Presentation)	ใช้เก็บข้อมูลการประมวลผล เช่น Key หลัก, ID, เลขรหัส
ความสำคัญของลำดับ	ลำดับการวางตำแหน่งมีความสำคัญ	ลำดับการเขียนไม่มีความสำคัญ

4. ทฤษฎีและคำสั้ง Document Object Model (DOM)

- DOM คือการจำลองเอกสาร XML/HTML ให้อยู่ในหน่วยความจำของ คอมพิวเตอร์ในลักษณะของโครงสร้างต้นไม้ (XML Node Tree).
- ประเภทของโหนด (Node Terminology):
 1. Document Node: โหนดสูงสุดที่ใช้ในการอ้างอิงถึงเอกสาร XML/HTML ทั้งเอกสาร.
 2. Element Node: โหนดที่ทำหน้าที่เป็นแท็กของเอกสาร.
 3. Attribute Node: โหนดคุณลักษณะที่อยู่ภายใต้ Element Node อื่นๆ.
 4. Text Node: โหนดที่ทำหน้าที่เก็บค่าข้อความ String จริง ๆ ที่ อยู่ภายในแท็ก.
- ความสัมพันธ์ของโหนด: Parent Node (โหนดพ่อ), Child Node (โหนด ลูก), Sibling Nodes (โหนดพี่น้องที่มีโหนดพ่อตัวเดียวกัน).

กับดักวิเคราะห์ข้อสอบ: การดึงค่า Text Node (ดึงค่า "Alan" จากแท็ก

<firstName>Alan</firstName>)

- หากเราเขียนเพียง xmlDoc.getElementsByTagName("firstName") เราจะเข้าถึงได้แค่ Element Node แต่เราต้องการเข้าถึงค่าที่เป็นสตริง จริง ๆ ซึ่งเป็นลูกของมัน.
- ไวยากรณ์ดึงค่าที่ถูกต้อง:
 - xmlDoc.getElementsByTagName("firstName").firstChild .nodeValue

- xmlDoc.getElementsByTagName("firstName").childNo des.nodeValue
- xmlDoc.getElementsByTagName("firstName").textContent (เป็นวิธีที่ง่ายและเร็วที่สุด)

โพร JavaScript API สำหรับ DOM Parsing และ Navigation:

- คำสั่ง Parse จาก String:

var parser = new DOMParser(); // [37, 38]

var xmlDoc = parser.parseFromString(text, "text/xml"); // [37, 38]

- xmlDoc.documentElement: คำสั่งอ้างอิงถึง Root Element ของ เอกสาร.
- xmlDoc.documentElement.nodeName: ดึงชื่อแท็กของ Root Element ออกมา.
- element.getAttribute("attr_name"): ดึงค่าแอตทริบิวต์ที่กำหนดภายใต้ Element นั้น ๆ.

CHAPTER 3.1 & 3.2: JSON & REST API

1. นิยามและโครงสร้างของ JSON (JavaScript Object Notation)

- JSON คือ รูปแบบการแลกเปลี่ยนข้อมูลประเภทน้ำหนักเบา (Lightweight data-interchange format) ที่กระชับ ไม่ใช่ Element รกและฟุ่มเฟือยเหมือน XML จึงทำงานได้เร็วกว่าและมีขนาดไฟล์เล็กกว่า.
- Language Independent: ทำงานเป็นอิสระและสนับสนุนการอ่านหรือ สร้างผ่านภาษาโปรแกรมที่หลากหลาย.
- กฎโครงสร้างไวยากรณ์: ข้อมูลต้องเก็บในรูปแบบ Key-Value pairs โดยที่ตัว Key (หรือ Property) จำเป็นต้องเขียนอยู่ภายใต้เครื่องหมาย อนุญาตคู่ "" เสมอ.
- ชนิดข้อมูลที่ JSON รองรับ (Supported Data Types): Strings, Numbers, Arrays, Booleans, Objects, และ null. (หมายเหตุ: JSON ไม่สนับสนุนประเภทข้อมูลฟังก์ชันหรือ Comment).

2. การเปรียบเทียบละเอียด: XML vs. JSON

- ส่วนที่เหมือนกัน: มนุษย์อ่านเข้าใจได้ (Human Readable), มีโครงสร้าง แบบลำดับชั้น (Hierarchical), สามารถ Parse ได้หลายภาษา, และ ประยุกต์ใช้ผ่าน XMLHttpRequest (AJAX) ได้ทั้งคู่.
- ส่วนที่แตกต่างกัน (Features Comparison):
 1. Data Storage: XML จัดเก็บในรูปแบบ Tree structure ขณะที่ JSON เก็บในลักษณะ Key-value pairs เชื่อมโยงกัน.
 2. Validation (Schema): XML มีโครงสร้างตรวจสอบความ ถูกต้องที่แข็งแกร่งผ่าน DTD และ XML Schema ส่วน JSON ไม่มีระบบตรวจสอบ Schema ในตัว.
 3. Array Support: JSON สนับสนุนโครงสร้าง Array โดยตรง (สัญญาณ []) ส่วน XML ไม่สนับสนุนโครงสร้างนี้โดยตรง (หาก ต้องการต้องใช้วิธีวนเขียนซ้ำ Elements เอง).
 4. Namespaces & Comments: XML สนับสนุนการตั้ง Namespace และเขียน Comment ได้อย่างสมบูรณ์ ส่วน JSON ไม่สนับสนุน Namespaces และไม่ยอมให้เขียน ข้อความ Comment ใด ๆ ในไฟล์.
 5. Parser: XML ต้องใช้ XML Parser เฉพาะเจาะจงเพื่อ ประมวลผล ขณะที่ JSON สามารถแกะข้อมูลโดยใช้ฟังก์ชัน มาตรฐานของภาษา JavaScript ได้ทันที.

3. เมื่อไรควรเลือกใช้ XML หรือ JSON?

- ควรเลือกใช้ XML เมื่อ:

1. งานต้องการการอธิบายข้อมูลที่ละเอียด ชัดเจน เพื่อความสะดวกในการสืบค้นข้อมูล เนื่องจากมีเครื่องมือช่วยที่แข็งแกร่ง เช่น XPath, XQuery, DOM, XSL.
2. งานต้องการการตรวจสอบความถูกต้องของโครงสร้างและเนื้อหาข้อมูลอย่างเข้มงวด (ใช้ DTD / Schema).
3. ต้องการระบบที่มีความปลอดภัยสูงมากผ่านโปรโตคอล SOAP ที่กำหนดรูปแบบผ่าน XML เท่านั้น.

- **ควรเลือกใช้ JSON เมื่อ:** ต้องการความรวดเร็วในการส่งข้อมูลผ่านเครือข่าย, งานประเภท AJAX Application หรือ Web Services น้ำหนักเบา (REST API) เนื่องจากขนาดของไฟล์ที่เล็กทำให้ลดปริมาณ Bandwidth และเพิ่มประสิทธิภาพในการทำงานของบราวเซอร์.

4. REST API และกลไก Client-Server

- **REST API:** ข้อมูลที่ส่งกลับมาจาก Server สู่ Client จะอยู่ในรูปแบบของ JSON ซึ่ง Client สามารถแกะข้อมูลดิบไปคำนวณและประมวลผลต่อได้ทันที โดยไม่ถูกบังคับสไลด์การแสดงผลเหมือนการส่งหน้า HTML สำเร็จรูป.
- **AJAX (XMLHttpRequest):** เป็นเทคโนโลยีฝั่ง Client ที่ช่วยส่ง HTTP Request ไปยัง Server เบื้องหลัง และรอรับผลลัพธ์ผ่าน Callback Function โดยไม่ต้องโหลดหน้าเว็บใหม่ทั้งหมด.
- **เปรียบเทียบได้ดแปลงข้อมูลดิบ (Data Conversion Gotcha):**
 - ในการดึงข้อมูลด้วย AJAX: ข้อมูลตอบกลับจะอยู่ในรูป JSON String บนตัวแปร responseText ต้องแปลงด้วย synchronous คำสั่ง:

```
const obj = JSON.parse(this.responseText); // [53, 56]
```

- ในการดึงข้อมูลด้วย **Fetch API:** ข้อมูลตอบกลับเป็นแบบ Stream ต้องเรียกใช้เมธอดแบบ asynchronous คืนค่าในรูปแบบ Promise:

```
fetch(apiUrl).then(function(response) { return response.json(); }) // [57]
```

CHAPTER 4: NOSQL DATABASES & MONGODB SCHEMA DESIGN

1. โครงสร้างและแนวคิดแบบจำลองความสัมพันธ์ (Data Relationships)

ใน NoSQL Database (MongoDB) ข้อมูลจัดเก็บเป็นรูปแบบเอกสาร (Document) จึงไม่มีตารางแบบ RDBMS ทำให้มีการออกแบบโครงสร้างหลัก ๆ 2 แบบ:

แบบที่ 1: Embedded Document Model (แบบฝังเอกสารซ้อนไว้)

ยุบรวมข้อมูลที่มีความสัมพันธ์กันไว้ในคอลเลกชันเดียว โดยนำข้อมูลอีกฝั่งมาฝังเป็นอีอบเจกต์ย่อยหรืออาร์เรย์ (Array of Objects).

- **ตัวอย่าง 1:1:** ยุบรวมข้อมูล Student และ PermanentAddress ไว้ด้วยกัน.
- **ตัวอย่าง 1:M:** ฝังข้อมูลที่อยู่หลาย ๆ แห่งของนักศึกษาลงไปเป็นฟิลด์อาร์เรย์ "address": [...].
- **ข้อดี:** เขียน Query ค้นหาข้อมูลได้ง่ายในขอบเขตอย่างรวดเร็ว และไม่ต้องทำงานผ่านการ Join ข้อมูลข้ามตาราง (No Join).
- **ข้อเสีย:** ขนาดเอกสารอาจโตขึ้นจนเกินขีดจำกัดสูงสุด และไม่เหมาะสมกรณีข้อมูลมีความซับซ้อนและมีการอัปเดตบ่อยครั้ง.

แบบที่ 2: Reference Document Model (แบบเก็บอ้างอิง ID ของโหนดปลายทาง) แยกเอกสารออกเป็นคนละคอลเลกชันอย่างอิสระ แล้วใช้การบันทึก ID ของคอลเลกชันอื่นเพื่อนำไปเชื่อมโยงกัน.

- **ข้อดี:** ข้อมูลเป็นอิสระต่อกัน ขนาดไฟล์ไม่โตเกินความจำเป็น.
- **ข้อเสีย:** การสืบค้นข้อมูลที่มีความเชื่อมโยงกันจะยากและซับซ้อน จำเป็นต้องทำ Left Outer Join ผ่านคำสั่ง \$lookup.

2. รูปแบบความสัมพันธ์แบบ Many-to-Many (M:N) ใน MongoDB

ในกรณีความสัมพันธ์แบบ Many-to-Many (เช่น นักศึกษา S00001 สามารถลงทะเบียนเรียนวิชา CRS101, CRS102 ได้ และหนึ่งวิชาเรียนมีนักศึกษได้หลายคน) สไลด์ระบุว่าสามารถออกแบบฐานข้อมูลได้ 3 รูปแบบหลัก ดังนี้:

รูปแบบที่ 1: ฝังข้อมูลการลงทะเบียน (Enrollments) ไว้ในคอลเลกชัน Student (Embedded Enrollments)

```
// Students Collection
```

```
{
  "_id": ObjectId("..."),
  "studentID": "S00001",
  "studentName": "Willy",
  "enrollments": [
    { "courseID": "CRS101", "semester": 1, "year": 2026, "grade": "A" },
    { "courseID": "CRS102", "semester": 1, "year": 2026, "grade": "B+" }
  ]
}
```

- **กรณีที่เหมาะสมกับการใช้งาน:** เหมาะกับแอปพลิเคชันที่ต้องดึงประวัติ **นักศึกษา, ผลการเรียน (Transcript) หรือข้อมูลการศึกษาของเด็กแต่ละคนมาแสดงบ่อยครั้ง** ดึงข้อมูลแค่ Document เดียวก็ได้ประวัติทั้งหมดโดยไม่ต้องทำ Join ข้อมูลมากมาย.
- **หมายเหตุ:** สไลด์ระบุว่าควรใช้วิธี Reference ไปยังคอลเลกชัน Course หลักด้วย เพื่อให้สามารถดึงข้อมูลรายวิชาที่เป็นปัจจุบันที่สุดได้ และสะดวกต่อการแก้ไขชื่อรายวิชา ณ จุดเดียว (แก้ปัญหารายการข้อมูลซ้ำซ้อน).
- **ข้อเสีย:** หากต้องการ Query ค้นหาว่า **"ในวิชานี้ มีนักศึกษาคนใดลงทะเบียนเรียนบ้าง"** จะทำได้ยากมาก เนื่องจากต้องไล่เปิดตรวจเช็คแทรกในทุก ๆ เอกสารของนักศึกษาทุกคน.

รูปแบบที่ 2: ฝังข้อมูลนักศึกษาไว้ในฝั่งวิชาเรียน (Embedded Students in Course) (เป็นรูปแบบที่กลับสลับขั้วกับรูปแบบแรก โดยประเมินความเร็วเมื่อต้องการวิเคราะห์รายวิชาเป็นหลัก).

รูปแบบที่ 3: สร้าง Enrollments Collection แยกออกจากกันเป็นตารางเชื่อมโยง (Reference Model)

แยกข้อมูลออกเป็นคอลเลกชันใหม่เพื่ออ้างอิงระหว่างวิชาเรียนและตัวนักศึกษา:

```
// Enrollments Collection
```

```
{
  "_id": ObjectId("..."),
  "courseID": "CRS102",
  "studentID": "S00001",
  "semester": 1,
  "year": 2026,
  "grade": "B+"
}
```

- **ข้อดี:** สามารถ Query ได้ทั้งสองแนวทาง คือ "ค้นหานักศึกษาเพื่อดูว่าลงทะเบียนวิชาใดบ้าง" หรือ "ค้นหาว่ารายวิชานี้มีนักศึกษาคนใดมาลงเรียนบ้าง".
- **ข้อเสีย:** การดึงข้อมูลมาแสดงผลจะต้องเรียกใช้ Stage **\$lookup** ในการ **Join ข้อมูลระหว่างคอลเลกชัน** ทำให้การประมวลผลซับซ้อนและต้องแลกมาด้วยประสิทธิภาพความเร็วที่ลดลง.

3. โครงสร้างคำสั่งสืบค้น (Aggregation Pipeline Stages)

- **\$match:** ทำหน้าที่กรองข้อมูลเฉพาะเอกสารที่ผ่านเงื่อนไขที่กำหนด.

- **\$project**: เลือกฟิลด์ข้อมูลที่ต้องการนำไปแสดงผลต่อ (กำหนดเป็น 1 เพื่อแสดง และ 0 เพื่อซ่อน) รวมถึงแปลงชื่อคอลัมน์ หรือนำฟิลด์มาคำนวณสูตรใหม่.
- **\$unwind**: ทำหน้าที่สลายข้อมูลอาร์เรย์ (Deconstruct Array Field) เพื่อทำการคลี่สมาชิกที่ซ่อนอยู่ในอาร์เรย์ ออกมาสร้างเป็นข้อมูลเอกสารเดี่ยวขึ้นใหม่แยกกันในท่อประมวลผล.
- **\$lookup**: ทำการเชื่อมโยงและผสานข้อมูลแบบ Left Outer Join เข้าไปยังคอลเลกชันเป้าหมาย.

o **ไวยากรณ์โครงสร้าง**:

```
{
  $lookup: {
    from: "ชื่อคอลเลกชันปลายทาง",
    localField: "ฟิลด์ปัจจุบันที่ใช้เทียบเคียง",
    foreignField: "ฟิลด์ของคอลเลกชันเป้าหมายที่จะจับคู่",
    as: "ชื่อฟิลด์อาร์เรย์ใหม่ที่จะนำไปเก็บอ็อบเจกต์ที่ดึงมา"
  }
}
```

CHAPTER 5: NODE.JS & NON-BLOCKING I/O MODEL

1. โครงสร้างสถาปัตยกรรม Tech Stack ในปัจจุบัน

- **Front-end (Browser)**: ใช้สำหรับทำงานติดต่อกับผู้ใช้ เช่น HTML, CSS และ JavaScript ทั่วไป รวมถึงการนำ Framework/Library ต่าง ๆ มาใช้งานเพื่อเพิ่มความเร็วในการเขียนโค้ดและแกะชิ้นส่วนกลับมาใช้งานซ้ำได้ง่าย เช่น **AngularJS** (ใช้ภาษา TypeScript เหมาะกับโปรเจกต์ขนาดใหญ่ พัฒนาโดย Google), **ReactJS** (พัฒนาโดย Facebook มีเครื่องมือสนับสนุนมาก), และ **VueJS**.
- **Back-end (Server)**: มีสภาพแวดล้อมที่รองรับการทำงานของ JavaScript ผังเวิร์กเวอร์ด้วยเทคโนโลยี **Node.js**.

2. สภาพแวดล้อมระบบการทำงานของ Node.js

- เมื่อทำการติดตั้ง Node.js ระบบจะทำการติดตั้งตัวแปลชุดคำสั่ง **V8 Engine (พัฒนาด้วยภาษา C++)** ของ Google มาให้ด้วย เพื่อทำหน้าที่ปรับปรุงและแปลงคำสั่งภาษา JavaScript ให้กลายเป็น **ภาษาเครื่อง (Machine Code)** ที่ระบบเวิร์กเวอร์เข้าใจโดยไม่จำเป็นต้องทำงานภายใต้ Web Browser อีกต่อไป.
- **ข้อดีเด่น**:
 1. นักพัฒนาสามารถเรียนรู้ศึกษาเพียงภาษาเดียว (**JavaScript**) ก็จะสามารถเขียนระบบเว็บแอปพลิเคชันได้ทั้งระบบหน้าบ้าน (Frontend) และระบบหลังบ้าน (Backend) ในฐานะของ **Full Stack Developer**.
 2. สามารถแชร์โค้ด JavaScript ระหว่าง Frontend และ Backend ร่วมกันได้โดยง่าย.
 3. มีกลุ่มชุมชนผู้ใช้ขนาดใหญ่ (Community) และคลังชุดเครื่องมือภายนอกสนับสนุน (**Third-Party Packages/npm**) จำนวนมหาศาล.
- **ความสามารถพิเศษฝั่ง Server**: สามารถทำงานเขียน-อ่านไฟล์ในเครื่องได้ (Read & Write files), สามารถเชื่อมต่อฐานข้อมูลได้ และประมวลผลเป็นเว็บเวิร์กเวอร์แบบเบ็ดเสร็จ.

- **กลไกทำงาน**: คำสั่งประมวลผลจะไหลไปที่ละลำดับบรรทัดตามแผนผังแรกไปจบถึงคำสั่งสุดท้าย โดยที่**จะต้องรันชุดคำสั่งด้านบนให้ทำงานจนเสร็จสมบูรณ์เสียก่อน จึงจะสามารถก้าวผ่านไปยังบรรทัดถัดไปได้**.
- **จุดเสีย**: เมื่อโปรแกรมเจอคำสั่งทำงานทางด้าน Input/Output (I/O) เช่น การอ่าน-เขียนไฟล์ หรือการส่งคำร้องดึงข้อมูลผ่านระบบอินเทอร์เน็ตที่ทำงานนอก CPU ตัวระบบประมวลผล CPU จำเป็นต้องหยุดทำงานเพื่อรอผลลัพธ์จาก I/O กลับมาเป็นเวลานาน ทำให้ระบบช้าและประสิทธิภาพต่ำ.

3.2 Asynchronous (Non-Blocking Model) ของ Node.js

- ชุดคำสั่ง JavaScript ภายใต้เทคโนโลยี Node.js ทำงานแบบ Non-blocking หรือ Asynchronous เพื่อแก้ปัญหาเวลาการหยุดค้างรอประมวลผลของบางขั้นตอนที่ใช้เวลายาวนาน.
- **กลไกทำงาน**: เมื่อระบบโปรแกรมประมวลผลเจอคำสั่งที่เกี่ยวข้องกับงาน I/O ระบบคอมพิวเตอร์จะส่งส่งผ่านงานเหล่านั้นไปรันอยู่เบื้องหลัง (**Background**) แล้วแจ้งตัวประมวลผล CPU ให้ข้ามผ่านไปจัดการประมวลผลคำสั่งถัดไปได้ทันทีโดยไม่ต้องรอผลตอบกลับจาก I/O ในจังหวะนั้น.
- **สูตรการคำนวณหาเวลารอผลลัพธ์เฉลี่ย (ออกสอบทฤษฎีคำนวณบ่อยมาก)**:

- o **กรณีที่ 1**: งานทั้งหมดที่ส่งมาประมวลผลเปรียบเทียบเป็นงานรูปแบบ **I/O Workloads** ทั้งหมด:

- **เวลารวมทั้งหมดที่รอ (Total Wait Time) = "เวลานานที่สุดของงานชิ้นใดงานชิ้นหนึ่งเท่านั้น"** (เพราะงานอื่นประมวลผลไปพร้อมกันในเบื้องหลังได้).

- o **กรณีที่ 2**: งานที่ส่งมีส่วนผสมทั้งงาน I/O และงานที่ต้องประมวลผลด้วย CPU (CPU-Based Workload) เช่น งานชิ้นที่ 1 เป็น I/O รัน 20 วินาที แต่งานที่ 2 ถึง 4 เป็นงาน CPU รัน 7, 10, และ 8 วินาทีตามลำดับ:

- **เวลารวมทั้งหมดที่รอ (Total Wait Time) = ผลรวมของงาน CPU ทั้งหมดครบเข้าด้วยกัน (7 + 10 + 8 = 25 วินาที) ซึ่งเวลานี้จะมากกว่าเวลารองานฝั่ง I/O 20 วินาทีไปเรียบร้อยแล้ว.**

3.3 ระบบรันไทม์เซิร์ฟเวอร์: Single-Threaded vs. Multi-Threaded

- **Single-Threaded Server**: ตัวรันไทม์เซิร์ฟเวอร์ของ Node.js จะออกแบบการทำงานผ่าน Thread ตัวเดียว เปรียบเสมือนการทำงานแบบพนักงานหนึ่งคนทำงานจัดการรับคำสั่งและส่งต่อด้วยสปีดที่เร็วมากโดยอาศัย Non-blocking I/O เข้ามาช่วย ไม่ให้เกิดจุดคอขวดที่พนักงานต้องยืนรอส่งงาน.
- **Multi-Threaded Server**: เซิร์ฟเวอร์ทั่วไปเปรียบเสมือนการมีพนักงานให้บริการรับคำขอได้มากกว่า 1 คนพร้อมกันเพื่อช่วยประมวลผลกระจายคำขอ

3. แบบจำลองสลับโหมดการทำงาน: Blocking vs. Non-Blocking I/O

3.1 Synchronous (Blocking Model)

- ภาษาโปรแกรมมีส่วนใหญ่ทำงานแบบ Synchronous หรือ Blocking.